UNIVERZITET U ZENICI

Politehnički fakultet

Odsjek softversko inženjerstvo

Otuzbir Hadžo

UPRAVLJANJE RESURSIMA I NJIHOVA ALOKACIJA U OBLAKU

Master rad

Mentor: Prof. dr. Đulaga Hadžić

Zenica, 2026.

BIBLIOGRAFSKA KARTICA RADA

NAUČNO PODRUČJE: tehničke nauke

NAUČNO POLJE: računarstvo i informatika

NAUČNA GRANA: softversko inženjerstvo

USTANOVA U KOJOJ JE RAD IZRAĐEN: Univerzitet u Zenici, Politehnički fakultet

ODSJEK: Softversko inženjerstvo

MENTOR RADA: prof. dr. Đulaga Hadžić

KOMENTOR RADA: —

DATUM ODBRANE RADA: _____

ČLANOVI KOMISIJE ZA ODBRANU: _____

SAŽETAK

Upravljanje resursima i njihova alokacija određuju performanse, trošak i efikasnost cloud usluga. Cilj rada je sistematizovati ključne politike upravljanja resursima i eksperimentalno ispitati odnos aktiviranog kapaciteta i kvaliteta usluge pri statičkoj alokaciji, CPU HPA i KEDA skaliranju.

Istraživanje povezuje analitičku, deskriptivnu, sintetičku i komparativnu metodu sa eksperimentalnom metodom. Razvijena je kontejnerizovana aplikacija za asinhronu batch obradu slika, mjerni postupak zasnovan na eksperimentalnim sesijama i automatizovana matrica opterećenja. Konačna evaluacija obuhvatila je statičku alokaciju, CPU HPA i KEDA skaliranje prema dužini reda, četiri profila opterećenja i pet ponavljanja svake kombinacije.

U 60 validnih izvođenja poslano je i završeno 67.954 posla bez greške. Dinamičke politike smanjile su aktivirani worker-kapacitet u svim profilima, od 14,7% do 83,4% u odnosu na statičku alokaciju. Najveća ušteda ostvarena je KEDA politikom pri malom opterećenju, bez pogoršanja vremena obrta. Pri trajnom i naglom visokom opterećenju smanjenje kapaciteta povećalo je čekanje, što potvrđuje kompromis između efikasnosti resursa i repne latencije.

Ključne riječi: cloud computing, upravljanje resursima, alokacija, autoscaling, Kubernetes, eksperimentalna evaluacija

ABSTRACT

Resource management and allocation determine the performance, cost, and efficiency of cloud services. This thesis systematizes the principal resource-management policies and experimentally examines the relationship between activated capacity and quality of service under static allocation, CPU-based HPA, and KEDA scaling.

The research combines analytical, descriptive, synthetic, comparative, and experimental methods. A containerized application for asynchronous batch image processing, a session-based measurement process, and an automated workload matrix were developed. The final evaluation covered static allocation, CPU-based horizontal scaling, queue-based KEDA scaling, four workload profiles, and five repetitions of every combination.

Across 60 valid runs, 67,954 jobs were submitted and completed without failure. The dynamic policies reduced activated worker capacity in every profile by 14.7% to 83.4% relative to static allocation. Queue-based KEDA achieved the largest reduction under low load without degrading turnaround time. Under sustained and sudden high load, lower initial capacity increased queueing, demonstrating the trade-off between resource efficiency and tail latency.

Keywords: cloud computing, resource management, allocation, autoscaling, Kubernetes, experimental evaluation

SADRŽAJ

UVOD.....11

1. Metodološki okvir istraživanja.....14

1.1. Predmet istraživanja.....14

1.2. Problem istraživanja.....14

1.3. Ciljevi istraživanja.....15

1.4. Hipoteze istraživanja.....15

1.5. Metode istraživanja.....16

1.6. Očekivani naučni i stručni doprinos rada.....17

1.7. Struktura rada.....17

2. Teorijske osnove cloud computinga.....19

2.1. Pojam i razvoj cloud computinga.....19

2.2. Osnovne karakteristike cloud okruženja.....20

2.3. Modeli usluga u cloud computingu.....21

2.3.1. Infrastruktura kao usluga (IaaS).....21

2.3.2. Platforma kao usluga (PaaS).....22

2.3.3. Softver kao usluga (SaaS).....22

2.4. Modeli implementacije cloud computinga.....23

2.4.1. Javni oblak.....23

2.4.2. Privatni oblak.....23

2.4.3. Hibridni oblak.....23

2.4.4. Community cloud.....24

2.5. Virtualizacija kao osnova cloud infrastrukture.....24

3. Upravljanje resursima u oblaku.....26

3.1. Pojam i značaj upravljanja resursima u oblaku.....26

3.2. Vrste resursa u cloud okruženju.....28

3.3. Izazovi upravljanja resursima u oblaku.....30

3.4. Odnos između performansi, funkcionalnosti i cijene.....32

3.5. Energetska efikasnost u cloud okruženju.....33
4. Alokacija resursa i scheduling mehanizmi u oblaku.....35
4.1. Pojam alokacije resursa u cloud okruženju.....35
4.2. Politike upravljanja resursima.....36
4.3. Planiranje i raspoređivanje zadataka u oblaku.....36
4.4. Preemptivno i nepreemptivno planiranje.....38
4.5. Prioritetno planiranje i raspoređivanje.....39
4.6. Fair queuing i deadline scheduling.....40
4.7. Algoritmi i modeli alokacije resursa.....40
4.8. Upravljanje resursima kao osnova optimizacije cloud sistema.....42
5. Dinamičko skaliranje i optimizacija resursa u oblaku.....43
5.1. Pojam i značaj dinamičkog skaliranja.....44
5.2. Horizontalno i vertikalno skaliranje.....47
5.3. Automatizovano upravljanje resursima.....51
5.4. Maksimiziranje protoka sistema.....54
5.5. Minimiziranje vremena obrta.....55
5.6. Optimizacija troškova korištenja cloud resursa.....56
5.7. Smanjenje potrošnje energije u cloud okruženju.....57
6. Analiza uticaja upravljanja resursima i njihove alokacije u oblaku.....59
6.1. Uticaj kvalitetnog upravljanja resursima na troškove cloud usluga.....59
6.2. Uticaj kvalitetne alokacije resursa na efikasnost korištenja IT resursa.....61
6.3. Komparativna analiza različitih pristupa upravljanju resursima.....61
6.4. Razmatranje hipoteza istraživanja.....63
6.5. Preporuke za efikasnije upravljanje resursima u oblaku.....64
7. Praktična evaluacija politika alokacije resursa.....66
7.1. Cilj evaluacije i istraživačka pitanja.....66
7.2. Sistem i eksperimentalno okruženje.....66
7.2.1. Konačno Kubernetes okruženje.....67

7.3. Eksperimentalne politike i profili opterećenja.....67
7.4. Metrike i veza sa hipotezama.....68
7.5. Postupak i kontrola valjanosti.....69
7.6. Lokalni pilot-test.....69
7.6.1. Svrha i ograničenja.....69
7.6.2. Rezultati pilota.....69
7.6.3. Tumačenje pilota.....70
7.6.4. Status hipoteza nakon pilota.....70
7.7. Konačni rezultati.....71
7.7.1. Potpunost skupa podataka.....71
7.7.2. Aktivirani kapacitet.....71
7.7.3. Kvalitet usluge.....72
7.7.4. Dinamika skaliranja.....74
7.7.5. Odnos kapaciteta i vremena obrta.....77
7.8. Ocjena hipoteza.....78
7.9. Prijetnje valjanosti.....78
7.10. Reproduktivnost.....79
ZAKLJUČAK.....80
LITERATURA.....81
DODATAK A: Prijedlog anketnog upitnika.....I

POPIS SLIKA

Slika 2.1. Karakteristike, modeli usluga i modeli implementacije cloud computinga……21

Slika 3.1. Zatvorena petlja automatizovanog upravljanja resursima.....28

Slika 3.2. Višekriterijska ravnoteža upravljanja resursima.....33

Slika 4.1. Pojednostavljeni tok scheduling odluke.....38

Slika 5.1. Horizontalno i vertikalno skaliranje.....51

Slika 5.2. Reaktivno i proaktivno skaliranje.....53

Slika 6.1. Ilustrativno poređenje potražnje i alociranog kapaciteta.....60

Slika 7.1. Aktivirani worker-kapacitet po profilu opterećenja.....72

Slika 7.2. Prosječno vrijeme obrta po profilu opterećenja.....73

Slika 7.3. Dinamika sistema tokom profila high.....75

Slika 7.4. Dinamika sistema tokom profila ramp.....76

Slika 7.5. Dinamika sistema tokom profila spike.....77

POPIS TABELA

Tabela P.1. Skraćenice korištene u radu.....10

Tabela 1.1. Veza ciljeva, metoda i očekivanih rezultata.....16

Tabela 2.1. Podjela odgovornosti u modelima cloud usluga.....22

Tabela 3.1. Resursi, tipične metrike i posljedice nedovoljne alokacije.....30

Tabela 4.1. Komparativni pregled scheduling pristupa.....41

Tabela 5.1. Poređenje pristupa autoscalingu.....54

Tabela 6.1. Ilustrativni proračun statičke i elastične alokacije.....60

Tabela 6.2. Komparativna analiza pristupa upravljanju.....63

Tabela 6.3. Teorijska ocjena istraživačkih hipoteza.....64

Tabela 7.1. Eksperimentalne politike alokacije worker replika.....67

Tabela 7.2. Profili eksperimentalnog opterećenja.....68

Tabela 7.3. Rezultati lokalnog pilot-testa.....69

Tabela 7.4. Aktivirani kapacitet i efikasnost workera.....71

Tabela 7.5. Vrijeme obrta, čekanje i throughput.....72

Tabela 7.6. Dinamika automatskog skaliranja.....74

Tabela 7.7. Eksperimentalna ocjena istraživačkih hipoteza.....78

POPIS SKRAĆENICA

Tabela P.1. Skraćenice korištene u radu

Skraćenica	Značenje
API	Application Programming Interface
CPU	Central Processing Unit
DRF	Dominant Resource Fairness
HPA	Horizontal Pod Autoscaler
IaaS	Infrastructure as a Service
PaaS	Platform as a Service
PUE	Power Usage Effectiveness
QoS	Quality of Service
SaaS	Software as a Service
SLA	Service Level Agreement
VM	Virtual Machine

Izvor: izrada autora.

UVOD

Razvoj informaciono-komunikacionih tehnologija u posljednjim decenijama doveo je do značajnih promjena u načinu projektovanja, implementacije i korištenja računarskih sistema. U tom kontekstu, računarstvo u oblaku predstavlja jedan od najznačajnijih savremenih modela pružanja informacionih usluga, jer omogućava pristup računarskim resursima na zahtjev, uz visok stepen fleksibilnosti, skalabilnosti i ekonomske opravdanosti. Zahvaljujući mogućnosti da se procesorska snaga, memorija, mrežni kapaciteti i skladišni resursi dijele među velikim brojem korisnika i aplikacija, cloud computing je postao nezaobilazan segment savremenog poslovanja, naučnih istraživanja i razvoja digitalnih usluga [2], [4], [5].

Jedno od ključnih pitanja u funkcionisanju cloud okruženja jeste upravljanje resursima i njihova pravilna alokacija. Efikasnost cloud sistema ne zavisi isključivo od raspoloživosti hardverskih i softverskih kapaciteta, nego u velikoj mjeri i od načina na koji se ti resursi raspoređuju, planiraju, nadziru i prilagođavaju dinamičkim zahtjevima korisnika i aplikacija. U uslovima velikog broja istovremenih zahtjeva, različitih prioriteta izvršavanja i promjenjivog opterećenja sistema, upravljanje resursima postaje složen i višedimenzionalan proces koji direktno utiče na performanse, pouzdanost, kvalitet usluge i ukupne troškove rada cloud infrastrukture [1], [7].

Planiranje i raspoređivanje resursa predstavlja kritičnu komponentu upravljanja resursima u oblaku, odgovornu za dijeljenje i multipleksiranje resursa na nekoliko nivoa sistema. Cloud okruženja najčešće podržavaju mješavinu različitih vrsta zadataka, pri čemu pojedini zadaci imaju stroga vremenska ograničenja, drugi nešto fleksibilnije zahtjeve, dok treći nemaju precizno određene rokove izvršenja. Upravo zbog toga sistemi planiranja i raspoređivanja resursa često se suočavaju sa kontradiktornim zahtjevima. S jedne strane, neophodno je osigurati visoke performanse i pravovremeno izvršavanje zadataka, a s druge strane minimizirati troškove, smanjiti potrošnju energije i postići što bolju iskorištenost raspoloživih resursa [1], [18].

Upravljanje resursima u oblaku može se posmatrati kroz dvije osnovne dimenzije. Prva dimenzija odnosi se na količinu resursa koja se dodjeljuje određenom korisniku, zadatku ili aplikaciji, dok se druga dimenzija odnosi na vrijeme i uslove pod kojima je pristup tim resursima odobren. Ove dimenzije neposredno utiču na osnovne kriterije ocjene cloud sistema, a to su performanse, funkcionalnost i cijena. Neadekvatna alokacija resursa može dovesti do zagušenja sistema, povećanja vremena odziva, lošije korisničke percepcije

kvaliteta usluge, smanjene iskorištenosti infrastrukture i rasta operativnih troškova. S druge strane, kvalitetno planiranje i upravljanje resursima omogućavaju ravnomjernije opterećenje sistema, bolji nivo dostupnosti usluga, veću efikasnost rada i racionalnije korištenje dostupnih kapaciteta.

Poseban značaj upravljanja resursima u oblaku ogleda se u tome što ono ne utiče samo na tehničke performanse sistema, nego i na ekonomsku održivost cloud usluga. Usljed povećanja obima podataka, broja korisnika i kompleksnosti aplikacija, organizacije koje koriste cloud tehnologije suočavaju se sa potrebom da istovremeno osiguraju visok kvalitet usluge i kontrolišu troškove korištenja infrastrukture. U tom smislu, kvalitetna alokacija resursa može doprinijeti smanjenju nepotrebnog rezervisanja kapaciteta, optimizaciji opterećenja, smanjenju potrošnje energije i efikasnijem korištenju IT resursa. Time upravljanje resursima postaje jedan od najvažnijih faktora za postizanje optimalnog odnosa između troškova i performansi u cloud okruženju [22].

Savremeni pristupi upravljanju resursima u oblaku sve više se zasnivaju na primjeni inteligentnih politika planiranja, automatizovanih mehanizama odlučivanja i dinamičkog skaliranja aplikacija. Dinamičko skaliranje omogućava prilagođavanje količine raspoloživih resursa stvarnim potrebama sistema u datom trenutku, čime se izbjegava i nedovoljna i prekomjerna alokacija. Pored toga, različiti algoritmi za sistemsku i mrežnu administraciju, kao i modeli raspoređivanja zadataka, imaju važnu ulogu u maksimiziranju protoka sistema, odnosno broja poslova koji se mogu završiti u određenoj jedinici vremena, te minimiziranju vremena obrta, odnosno vremena između podnošenja posla i njegovog završetka. Upravo iz tog razloga pitanje upravljanja resursima i njihove alokacije u oblaku ima i teorijski i praktični značaj [7], [8].

Predmet istraživanja ovog rada jeste analiza upravljanja resursima i njihove alokacije u cloud okruženju, sa posebnim fokusom na politike, mehanizme i modele planiranja koji utiču na performanse sistema, troškove pružanja usluge i efikasnost korištenja IT resursa. Istraživanje će biti usmjereno na sagledavanje faktora koji utiču na efikasnije planiranje i upravljanje resursima, kao i na utvrđivanje načina na koji različiti pristupi alokaciji mogu unaprijediti rad cloud sistema.

Osnovni cilj rada jeste istražiti na koji način kvalitetno upravljanje resursima i njihova pravilna alokacija mogu doprinijeti minimizaciji troškova pružanja cloud usluga i unapređenju efikasnosti upotrebe IT resursa u oblaku. U skladu s tim, rad polazi od dvije osnovne hipoteze. Prva hipoteza polazi od pretpostavke da se kvalitetnim upravljanjem

resursima i njihovom alokacijom mogu minimizirati troškovi za pružanje usluge u oblaku. Druga hipoteza zasniva se na pretpostavci da kvalitetno upravljanje resursima i njihova alokacija mogu dovesti do unapređenja efikasnosti upotrebe IT resursa u oblaku.

U radu su primijenjene analitička, sintetička, induktivna, komparativna, deskriptivna i eksperimentalna metoda. Teorijska analiza dopunjena je lokalnim pilotom i konačnom Kubernetes evaluacijom sa 60 strogo validiranih izvođenja. Bilježeni su aktivirani kapacitet, protok, čekanje, vrijeme obrta, iskorištenost resursa, reakcija skaliranja i greške.

Naučni i stručni značaj ovog rada ogleda se u sistematizaciji saznanja o upravljanju resursima u oblaku, kao i u sagledavanju njihove povezanosti sa ekonomskim i tehničkim aspektima funkcionisanja cloud sistema. Očekuje se da rezultati istraživanja doprinesu boljem razumijevanju politika i mehanizama za upravljanje resursima, ukažu na značaj i prednosti dinamičkog skaliranja aplikacija, te pokažu na koji način odgovarajući algoritmi i modeli alokacije resursa mogu doprinijeti maksimiziranju protoka i minimiziranju vremena obrta. Na osnovu provedenog istraživanja bit će izvedeni zaključci i preporuke koje mogu poslužiti kao osnova za efikasnije planiranje i upravljanje resursima u cloud okruženju.

Rad najprije prikazuje teorijske osnove cloud computinga i upravljanja resursima, zatim analizira politike alokacije, scheduling i dinamičko skaliranje. Nakon teorijske i komparativne analize prikazani su pilot mjernog postupka i konačno eksperimentalno poređenje statičke, CPU HPA i KEDA politike.

1. Metodološki okvir istraživanja

1.1. Predmet istraživanja

Predmet istraživanja ovog magistarskog rada jeste upravljanje resursima i njihova alokacija u cloud okruženju, sa posebnim fokusom na politike, mehanizme i algoritme planiranja koji utiču na efikasnost korištenja IT resursa, performanse sistema i troškove pružanja cloud usluga. U savremenim cloud sistemima raspoloživi resursi, kao što su procesorsko vrijeme, memorija, mrežni kapaciteti i skladišni prostor, dijele se između većeg broja korisnika, aplikacija i procesa, zbog čega njihovo pravilno upravljanje predstavlja jedan od ključnih preduslova za stabilan i efikasan rad sistema.

Predmet istraživanja obuhvata teorijsku i praktičnu analizu načina na koji različiti modeli alokacije resursa, strategije raspoređivanja zadataka i mehanizmi dinamičkog skaliranja utiču na sposobnost cloud sistema da odgovori na promjenjive zahtjeve korisnika. Posebna pažnja bit će posvećena odnosu između kvaliteta upravljanja resursima i ekonomskih, tehničkih i funkcionalnih aspekata rada cloud infrastrukture. U tom smislu, istraživanje će obuhvatiti razmatranje politika upravljanja resursima, modela planiranja, metoda optimizacije opterećenja i pristupa koji doprinose boljoj iskorištenosti infrastrukture, smanjenju troškova i unapređenju ukupnih performansi sistema.

1.2. Problem istraživanja

Osnovni problem istraživanja proizlazi iz činjenice da cloud okruženja funkcionišu u uslovima dinamičkih i često nepredvidivih promjena opterećenja, pri čemu različiti korisnici i aplikacije postavljaju različite zahtjeve prema raspoloživim resursima. U takvim uslovima, neefikasna alokacija resursa može dovesti do smanjenja performansi sistema, povećanja vremena odziva, slabije iskorištenosti infrastrukture, rasta troškova i povećane potrošnje energije. S druge strane, pretjerano rezervisanje resursa može dovesti do neracionalnog korištenja kapaciteta i nepotrebnog povećanja operativnih troškova.

Problem istraživanja ogleda se, dakle, u potrebi da se utvrdi na koji način se raspoloživi resursi u oblaku mogu planirati i dodjeljivati tako da se postigne optimalan balans između performansi, cijene i efikasnosti korištenja infrastrukture. Ovaj problem dodatno je složen činjenicom da cloud sistemi često istovremeno podržavaju različite vrste zadataka, od onih koji imaju stroga vremenska ograničenja, do onih čije izvršavanje nije vremenski kritično. Zbog toga se nameće potreba za detaljnim proučavanjem politika upravljanja resursima i

mehanizama alokacije, kako bi se identifikovali pristupi koji omogućavaju efikasnije funkcionisanje cloud sistema.

1.3. Ciljevi istraživanja

Osnovni cilj istraživanja jeste analizirati značaj upravljanja resursima i njihove alokacije u cloud okruženju, te utvrditi u kojoj mjeri kvalitetni mehanizmi planiranja i raspoređivanja mogu doprinijeti smanjenju troškova i povećanju efikasnosti korištenja IT resursa.

Iz osnovnog cilja proizlaze sljedeći posebni ciljevi istraživanja:

- analizirati teorijske osnove cloud computinga i upravljanja resursima u oblaku;
- identifikovati ključne politike i mehanizme upravljanja resursima u cloud okruženju;
- razmotriti različite modele i algoritme alokacije resursa i planiranja zadataka;
- analizirati značaj dinamičkog skaliranja aplikacija u optimizaciji cloud sistema;
- utvrditi odnos između kvaliteta upravljanja resursima i troškova pružanja cloud usluga;
- ispitati na koji način pravilna alokacija resursa utiče na efikasnost korištenja IT resursa;
- ukazati na mogućnosti maksimiziranja protoka i minimiziranja vremena obrta pomoću odgovarajućih mehanizama planiranja i administracije;
- dati zaključke i preporuke za efikasnije upravljanje resursima u oblaku.

1.4. Hipoteze istraživanja

Polazeći od predmeta i ciljeva istraživanja, u radu su postavljene sljedeće hipoteze:

Hipoteza 1: Kvalitetnim upravljanjem resursima i njihovom alokacijom mogu se minimizirati troškovi za pružanje usluge u oblaku.

Hipoteza 2: Kvalitetnim upravljanjem resursima i njihovom alokacijom u oblaku može doći do unapređenja efikasnosti upotrebe IT resursa u oblaku.

Navedene hipoteze zasnivaju se na pretpostavci da pravilno planiranje resursa, primjena odgovarajućih politika raspoređivanja i korištenje dinamičkih mehanizama skaliranja omogućavaju racionalnije korištenje infrastrukture, bolju raspodjelu opterećenja i veći stepen iskorištenosti raspoloživih kapaciteta. Time se stvaraju uslovi za smanjenje troškova i poboljšanje ukupne efikasnosti cloud sistema.

1.5. Metode istraživanja

U izradi magistarskog rada primijenjeno je više naučnoistraživačkih metoda kako bi se predmet sagledao sa teorijskog i praktičnog aspekta.

Analitička metoda korištena je za razlaganje procesa upravljanja resursima na politike alokacije, algoritme raspoređivanja, metrike i mehanizme skaliranja.

Sintetička metoda primijenjena je za povezivanje pojedinačnih nalaza u cjelinu iz koje su izvedeni zaključci i preporuke.

Induktivna metoda korištena je za izvođenje općih zaključaka iz analiziranih pristupa i opažanja pilota i konačne eksperimentalne matrice, uz jasno navedena ograničenja njihovog opsega.

Komparativna metoda korištena je za poređenje politika i algoritama prema trošku, performansama i efikasnosti korištenja resursa.

Deskriptivna metoda primijenjena je za sistematičan prikaz pojmova, principa, arhitekture eksperimentalne aplikacije i izmjerenih pokazatelja.

Eksperimentalna metoda primijenjena je u lokalnom pilotu i kontrolisanoj Kubernetes matrici sa tri politike alokacije, četiri profila opterećenja i pet ponavljanja. Rezultati su analizirani deskriptivnim pokazateljima i vremenskim serijama. Teorijski dio zasnovan je na relevantnoj naučnoj i stručnoj literaturi.

Tabela 1.1. Veza ciljeva, metoda i očekivanih rezultata

Istraživački element	Primijenjena metoda	Očekivani rezultat
Sistematizacija pojmova	Deskriptivna i sintetička	Uređen teorijski okvir
Poređenje politika	Analitička i komparativna	Prednosti, ograničenja i područja primjene
Procjena hipoteza	Induktivna i komparativna	Uslovna ocjena uticaja na trošak i efikasnost
Ilustrativni scenarij	Analitički proračun	Ilustracija posljedica alokacije

Istraživački element	Primijenjena metoda	Očekivani rezultat
Lokalna pilot-evaluacija	Eksperimentalna i deskriptivno-statistička	Validacija mjernog postupka i kalibracija opterećenja

Izvor: izrada autora.

1.6. Očekivani naučni i stručni doprinos rada

Doprinos rada ogleda se u povezivanju teorijske sistematizacije upravljanja resursima sa reproducibilnim mjernim postupkom na kontejnerizovanoj aplikaciji. Rad razdvaja vrijeme obrade, čekanja i ukupnog obrta, te pokazuje zašto se tehnički i ekonomski pokazatelji moraju tumačiti zajedno.

Naučni doprinos čini analitičko i komparativno sagledavanje postojećih pristupa uz jasno operacionalizovane hipoteze. Stručni doprinos čine aplikacija, automatizovani eksperimentalni runner, kontrola valjanosti izvođenja i postupak agregacije rezultata koji se može primijeniti u punoj Kubernetes evaluaciji.

Očekuje se da rad:

- predstavi politike i mehanizme upravljanja resursima u oblaku;
- analizira značaj, moguće koristi i ograničenja dinamičkog skaliranja aplikacija;
- objasni kako odgovarajući algoritmi planiranja i administracije doprinose maksimiziranju protoka sistema;
- ukaže na mogućnosti minimiziranja vremena obrta;
- ispita uslove pod kojima kvalitetno upravljanje može smanjiti aktivirani kapacitet i unaprijediti efikasnost korištenja IT resursa.

1.7. Struktura rada

Magistarski rad strukturiran je tako da problem upravljanja resursima razmatra od teorijskih osnova do praktične provjere mjernog postupka.

U uvodnom dijelu predstavljeni su problem i predmet istraživanja, ciljevi, hipoteze, metode te naučni i stručni doprinos rada.

Drugo poglavlje obrađuje teorijske osnove cloud computinga, modele usluga i implementacije te ulogu virtualizacije.

Treće poglavlje razmatra upravljanje resursima, vrste resursa, glavne izazove i odnos performansi, funkcionalnosti i cijene.

Četvrto poglavlje obrađuje alokaciju resursa, politike raspoređivanja i scheduling mehanizme.

Peto poglavlje analizira dinamičko skaliranje, protok, vrijeme obrta, trošak i energetsku efikasnost.

Šesto poglavlje povezuje teorijske nalaze sa troškom i efikasnošću te daje uslovnu teorijsku ocjenu hipoteza.

Sedmo poglavlje prikazuje lokalni pilot, eksperimentalno okruženje, strogu kontrolu valjanosti i konačno poređenje statičke, CPU HPA i KEDA politike. Zaključak objedinjuje teorijske i praktične nalaze, ograničenja i preporuke za izbor politike.

2. Teorijske osnove cloud computinga

2.1. Pojam i razvoj cloud computinga

Računarstvo u oblaku predstavlja savremeni model pružanja računarskih usluga putem mreže, najčešće interneta, pri čemu korisnici mogu pristupati različitim hardverskim i softverskim resursima na zahtjev, bez potrebe da ih direktno posjeduju ili održavaju. U okviru ovog modela računarski resursi, kao što su procesorska snaga, memorija, prostor za pohranu podataka, razvojne platforme i aplikativni softver, isporučuju se kao standardizirane usluge koje se mogu koristiti fleksibilno, prema potrebama korisnika i organizacija [2], [4], [5].

Osnovna ideja cloud computinga zasniva se na tome da se informatički resursi ne posmatraju isključivo kao lokalna infrastruktura vezana za jednog korisnika ili jednu organizaciju, nego kao skup dijeljenih i virtualizovanih kapaciteta koji se mogu dinamički dodjeljivati i povlačiti u skladu sa trenutnim zahtjevima. Na taj način korisniku se omogućava da koristi resurse kada su mu potrebni, u obimu koji mu je potreban, pri čemu se naplata često vrši prema stvarnoj upotrebi [2], [5].

Razvoj cloud computinga usko je povezan sa razvojem distribuiranih sistema, virtualizacije, mrežnih tehnologija i podatkovnih centara. Prve ideje o dijeljenju računarskih resursa javljaju se još u periodu razvoja velikih centralnih računarskih sistema, kada je više korisnika pristupalo istom računarskom kapacitetu putem terminala. Kasnije su koncepti grid computinga, utility computinga i servisno orijentisane arhitekture doprinijeli oblikovanju osnove za nastanak računarstva u oblaku [4], [5].

Poseban razvojni zamah cloud computing dobija početkom 21. stoljeća, kada povećanje dostupnosti širokopojasnog interneta, napredak virtualizacijskih tehnologija i rast potreba za fleksibilnim IT rješenjima omogućavaju masovniju primjenu ovog modela. Organizacije su prepoznale da održavanje vlastite IT infrastrukture zahtijeva značajna finansijska ulaganja, stručne kadrove i kontinuirano tehnološko unapređenje, dok cloud model omogućava racionalnije korištenje resursa, veću prilagodljivost i lakšu dostupnost usluga [4].

Savremeni cloud computing danas predstavlja temelj brojnih informacionih sistema, poslovnih aplikacija, platformi za razvoj softvera, sistema za pohranu i obradu velikih količina podataka, kao i različitih digitalnih servisa koje koriste pojedinci, preduzeća, obrazovne ustanove i javna uprava. Njegova važnost ogleda se u tome što omogućava

organizacijama da se više fokusiraju na svoje osnovne poslovne procese, a manje na tehničke aspekte upravljanja infrastrukturom [4], [5].

2.2. Osnovne karakteristike cloud okruženja

Cloud computing karakteriše nekoliko osnovnih obilježja koja ga razlikuju od tradicionalnih modela korištenja IT infrastrukture. Te karakteristike predstavljaju osnovu njegove funkcionalnosti i razlog su široke primjene u savremenim informacionim sistemima [2].

Prva karakteristika jeste samouslužno korištenje resursa na zahtjev. Korisnici mogu samostalno pristupiti određenim uslugama, kao što su virtualni serveri, skladišni prostor ili aplikacije, bez potrebe za neposrednom intervencijom pružaoca usluge. Time se postiže veća brzina i fleksibilnost u korištenju resursa [2].

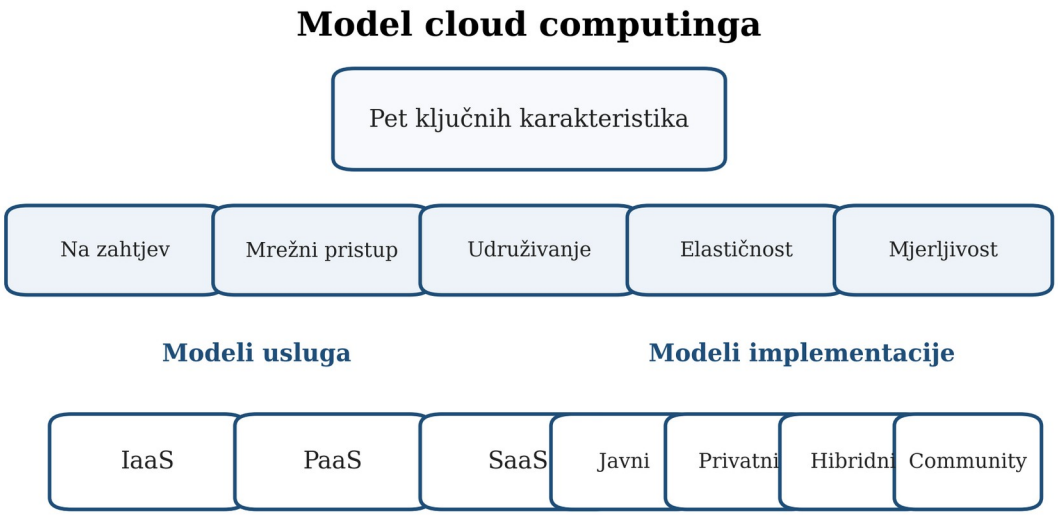
Druga karakteristika odnosi se na široku mrežnu dostupnost. Cloud usluge dostupne su putem standardnih mrežnih mehanizama i različitih uređaja, uključujući desktop računare, prijenosne računare, tablete i mobilne telefone. Ova karakteristika omogućava korisnicima pristup podacima i aplikacijama sa različitih lokacija i uređaja [2].

Treća karakteristika jeste udruživanje resursa. Resursi pružaoca cloud usluga objedinjeni su u jedinstvenu infrastrukturu koja služi većem broju korisnika prema modelu dijeljenja kapaciteta. Takav pristup omogućava efikasnije korištenje infrastrukture, jer se resursi dinamički raspoređuju u zavisnosti od potreba [2].

Četvrta karakteristika jeste brza elastičnost. Cloud sistemi omogućavaju brzo povećanje ili smanjenje količine dostupnih resursa u skladu sa promjenama opterećenja. Ova sposobnost posebno je važna u situacijama kada dolazi do naglog rasta ili pada broja korisničkih zahtjeva [2].

Peta važna karakteristika jeste mjerljivost usluge. Korištenje resursa u cloud okruženju može se pratiti, kontrolisati i analizirati, što omogućava transparentno upravljanje potrošnjom i često naplatu prema stvarnoj upotrebi. Ova karakteristika doprinosi boljoj kontroli troškova i racionalnijem korištenju IT kapaciteta [2].

Pored navedenih osobina, cloud okruženje odlikuju i skalabilnost, dostupnost, pouzdanost, geografska nezavisnost i mogućnost automatizovanog upravljanja resursima. Sve navedene karakteristike čine cloud computing pogodnim za organizacije koje zahtijevaju dinamično prilagođavanje resursa i efikasno upravljanje infrastrukturom [2], [3].



Slika 2.1. Karakteristike, modeli usluga i modeli implementacije cloud computinga

Izvor: izrada autora prema [2] i [3].

2.3. Modeli usluga u cloud computingu

Jedna od osnovnih podjela unutar cloud computinga odnosi se na modele usluga koje se korisnicima isporučuju putem oblaka. Ti modeli definišu nivo apstrakcije resursa koji se stavlja na raspolaganje korisniku, kao i stepen kontrole koji korisnik ima nad infrastrukturom, platformom i aplikacijama [2], [3].

2.3.1. Infrastruktura kao usluga (IaaS)

Infrastruktura kao usluga predstavlja model u kojem pružalac cloud usluge korisniku omogućava pristup osnovnim računarskim resursima, kao što su virtualne mašine, procesorski kapaciteti, memorija, mrežni resursi i skladišni prostor. Korisnik ima mogućnost da instalira operativne sisteme, aplikacije i druge softverske komponente prema vlastitim potrebama, dok pružalac usluge upravlja fizičkom infrastrukturom i osnovnim slojem virtualizacije [2].

Prednost ovog modela ogleda se u visokom stepenu fleksibilnosti i kontroli koju korisnik ima nad resursima. Organizacije mogu koristiti infrastrukturu bez potrebe za kupovinom i održavanjem vlastitih servera, čime se smanjuju početna ulaganja i olakšava skaliranje sistema. Ovaj model posebno je pogodan za razvojna okruženja, testiranje aplikacija, hosting servisa i sisteme sa promjenjivim opterećenjem.

2.3.2. Platforma kao usluga (PaaS)

Platforma kao usluga predstavlja model u kojem korisniku nije na raspolaganju samo infrastruktura, nego i kompletno razvojno i izvršno okruženje za izradu, testiranje, implementaciju i upravljanje aplikacijama. U okviru ovog modela pružalac usluge obezbjeđuje operativni sistem, baze podataka, razvojne alate, middleware i druge komponente potrebne za razvoj softvera [2].

Glavna prednost PaaS modela jeste u tome što korisnici i razvojni timovi mogu biti fokusirani na funkcionalnost aplikacije, bez potrebe da upravljaju nižim slojevima infrastrukture. Time se ubrzava razvoj softvera, pojednostavljuje implementacija i smanjuje složenost administracije sistema. Ovaj model je posebno koristan za razvoj web aplikacija, poslovnih sistema i servisa koji zahtijevaju brzo puštanje u rad.

2.3.3. Softver kao usluga (SaaS)

Softver kao usluga predstavlja model u kojem korisnici pristupaju gotovim aplikacijama putem interneta, bez potrebe za njihovom lokalnom instalacijom ili održavanjem. U ovom slučaju pružalac cloud usluge upravlja kompletnom infrastrukturom, platformom i samom aplikacijom, dok korisnik koristi funkcionalnosti aplikacije preko web preglednika ili druge klijentske aplikacije [2].

Prednosti SaaS modela ogledaju se u jednostavnosti korištenja, brzoj dostupnosti i smanjenju potrebe za tehničkim održavanjem na strani korisnika. Organizacije mogu koristiti poslovne, komunikacijske, obrazovne i druge aplikacije bez značajnih ulaganja u lokalnu infrastrukturu. Ovaj model posebno je pogodan za standardizirane softverske usluge kao što su elektronska pošta, upravljanje dokumentima, sistemi za saradnju i poslovne aplikacije.

Tabela 2.1. Podjela odgovornosti u modelima cloud usluga

Sloj	IaaS	PaaS	SaaS
Aplikacija i podaci	Korisnik	Korisnik	Pružalac usluge
Izvršno okruženje	Korisnik	Pružalac usluge	Pružalac usluge
Operativni sistem	Korisnik	Pružalac usluge	Pružalac usluge
Virtualizacija i hardver	Pružalac usluge	Pružalac usluge	Pružalac usluge

Izvor: izrada autora prema [2] i [3].

2.4. Modeli implementacije cloud computinga

Pored modela usluga, cloud computing se može razlikovati i prema načinu implementacije, odnosno prema tome ko koristi infrastrukturu, ko njome upravlja i na koji način je organizovan pristup resursima [2], [3].

2.4.1. Javni oblak

Javni oblak predstavlja model u kojem cloud infrastrukturu posjeduje, održava i pruža treća strana, a usluge su dostupne širokom krugu korisnika putem interneta. U ovom modelu više korisnika koristi zajedničku infrastrukturu, pri čemu se resursi dodjeljuju prema potrebama i ugovorenim uslovima korištenja [2].

Prednosti javnog oblaka ogledaju se u nižim troškovima, visokoj skalabilnosti i jednostavnom pristupu uslugama. Međutim, zbog dijeljenja infrastrukture i manjeg stepena direktne kontrole, pojedine organizacije mogu imati određene rezerve kada su u pitanju sigurnost, privatnost i usklađenost sa regulatornim zahtjevima.

2.4.2. Privatni oblak

Privatni oblak podrazumijeva cloud infrastrukturu namijenjenu isključivo jednoj organizaciji. Njome može upravljati sama organizacija ili vanjski pružalac usluge, a može biti smještena unutar organizacije ili na eksternoj lokaciji. Ovaj model omogućava viši stepen kontrole, sigurnosti i prilagođavanja specifičnim potrebama korisnika [2].

Privatni oblak posebno je pogodan za organizacije koje raspolažu osjetljivim podacima, imaju stroge zahtjeve u pogledu sigurnosti ili žele zadržati veći nivo kontrole nad svojom infrastrukturom. Ipak, njegovo uspostavljanje i održavanje često zahtijeva veća ulaganja u odnosu na javni oblak.

2.4.3. Hibridni oblak

Hibridni oblak predstavlja kombinaciju dva ili više cloud modela, najčešće javnog i privatnog oblaka, koji ostaju zasebni entiteti, ali su povezani tako da omogućavaju razmjenu podataka i aplikacija. Ovaj model organizacijama omogućava da dio opterećenja i osjetljivih procesa zadrže u privatnom oblaku, dok manje osjetljive ili promjenjive zahtjeve mogu preusmjeriti na javni oblak [2].

Prednost hibridnog oblaka ogleda se u fleksibilnosti, jer omogućava optimalno kombinovanje sigurnosti i kontrole sa skalabilnošću i ekonomičnošću. Takav pristup je posebno koristan za organizacije koje žele postepenu migraciju u cloud ili imaju promjenjive potrebe za resursima.

2.4.4. Community cloud

Community cloud predstavlja model u kojem infrastrukturu zajednički koristi više organizacija koje imaju slične interese, zahtjeve ili regulatorne obaveze. Takve organizacije mogu pripadati istoj djelatnosti, sektoru ili administrativnom sistemu, pa zajednički koriste cloud infrastrukturu prilagođenu njihovim potrebama [2].

Ovaj model omogućava dijeljenje troškova i resursa između organizacija koje imaju slične sigurnosne, funkcionalne ili pravne zahtjeve. Iako nije toliko zastupljen kao javni, privatni ili hibridni oblak, community cloud može biti veoma koristan u oblastima kao što su obrazovanje, zdravstvo i javna uprava.

2.5. Virtualizacija kao osnova cloud infrastrukture

Jedan od najvažnijih tehnoloških temelja cloud computinga jeste virtualizacija. Virtualizacija predstavlja proces stvaranja virtualnih verzija računarskih resursa, kao što su serveri, operativni sistemi, skladišni uređaji ili mrežni elementi, pri čemu se fizički resursi apstrahuju i dijele na više logičkih cjelina koje se mogu nezavisno koristiti i upravljati [1], [5].

U kontekstu cloud computinga virtualizacija omogućava da se fizička infrastruktura koristi mnogo efikasnije nego u tradicionalnim okruženjima. Na jednom fizičkom serveru može se pokrenuti više virtualnih mašina, pri čemu svaka od njih funkcioniše kao zaseban računarski sistem sa vlastitim operativnim sistemom, aplikacijama i resursima. Time se postiže bolja iskorištenost hardvera, smanjuje potreba za dodatnom opremom i omogućava jednostavnije upravljanje resursima.

Poseban značaj virtualizacije ogleda se u fleksibilnosti i brzini dodjele resursa. Virtualne mašine mogu se kreirati, premještati, zaustavljati ili skalirati u skladu sa promjenama opterećenja sistema, što cloud okruženju daje potrebnu elastičnost. Također, virtualizacija olakšava izolaciju korisnika i aplikacija, povećava sigurnost, pojednostavljuje oporavak sistema i doprinosi većoj pouzdanosti infrastrukture.

Pored virtualizacije servera, važnu ulogu imaju i virtualizacija mreže i virtualizacija skladišta, koje omogućavaju fleksibilnije upravljanje mrežnim tokovima i pohranom podataka. U novije

vrijeme značajnu ulogu ima i kontejnerizacija, kao lakši oblik virtualizacije, koja omogućava efikasno pakovanje i izvršavanje aplikacija uz manju potrošnju resursa [14].

Zahvaljujući virtualizaciji, cloud computing je dobio tehničku osnovu za dinamičko upravljanje kapacitetima, brzo skaliranje i efikasno raspoređivanje resursa. Upravo zbog toga virtualizacija predstavlja jednu od ključnih pretpostavki za razvoj i funkcionisanje savremenih cloud sistema [1], [5].

3. Upravljanje resursima u oblaku

3.1. Pojam i značaj upravljanja resursima u oblaku

Upravljanje resursima u oblaku predstavlja jedan od najvažnijih funkcionalnih aspekata cloud computinga, jer omogućava da se raspoloživi računarski kapaciteti koriste na efikasan, fleksibilan i ekonomski opravdan način. U cloud okruženju resursi nisu vezani isključivo za jednog korisnika, jednu aplikaciju ili jednu fizičku mašinu, već se posmatraju kao skup dijeljenih kapaciteta koji se dinamički dodjeljuju različitim zahtjevima u skladu sa trenutnim potrebama sistema. Zbog toga upravljanje resursima podrazumijeva kontinuiran proces planiranja, raspoređivanja, nadzora, prilagođavanja i optimizacije korištenja procesorske snage, memorije, mrežnih kapaciteta, skladišnog prostora i drugih infrastrukturnih elemenata [1].

U tradicionalnim informacionim sistemima resursi su često bili statički rezervisani za određene aplikacije ili korisnike, što je u mnogim slučajevima dovodilo do slabe iskorištenosti infrastrukture i većih troškova održavanja. Nasuprot tome, cloud computing se zasniva na principu dijeljenja i dinamičke raspodjele resursa, gdje se kapaciteti mogu prilagođavati promjenama opterećenja i zahtjevima korisnika gotovo u realnom vremenu. Upravo ta mogućnost dinamičke raspodjele čini upravljanje resursima centralnim pitanjem funkcionisanja cloud sistema.

Značaj upravljanja resursima ogleda se prije svega u njegovom direktnom uticaju na performanse sistema. Način na koji se resursi dodjeljuju korisnicima i aplikacijama utiče na brzinu izvršavanja zadataka, vrijeme odziva sistema, raspoloživost usluge i sposobnost sistema da obradi veliki broj istovremenih zahtjeva. Ukoliko upravljanje resursima nije adekvatno organizovano, mogu se javiti problemi kao što su preopterećenje pojedinih komponenti, zagušenje sistema, produženje vremena čekanja, neefikasno korištenje kapaciteta i smanjenje kvaliteta pružene usluge.

Pored uticaja na performanse, upravljanje resursima ima i snažnu ekonomsku dimenziju. Budući da cloud model podrazumijeva korištenje infrastrukture kao usluge, troškovi su usko povezani sa količinom zauzetih i iskorištenih resursa. Nepravilna alokacija može dovesti do nepotrebnog rezervisanja kapaciteta, čime se povećavaju troškovi, dok nedovoljna alokacija može izazvati pad performansi i posredno dovesti do dodatnih gubitaka. Zbog toga efikasno

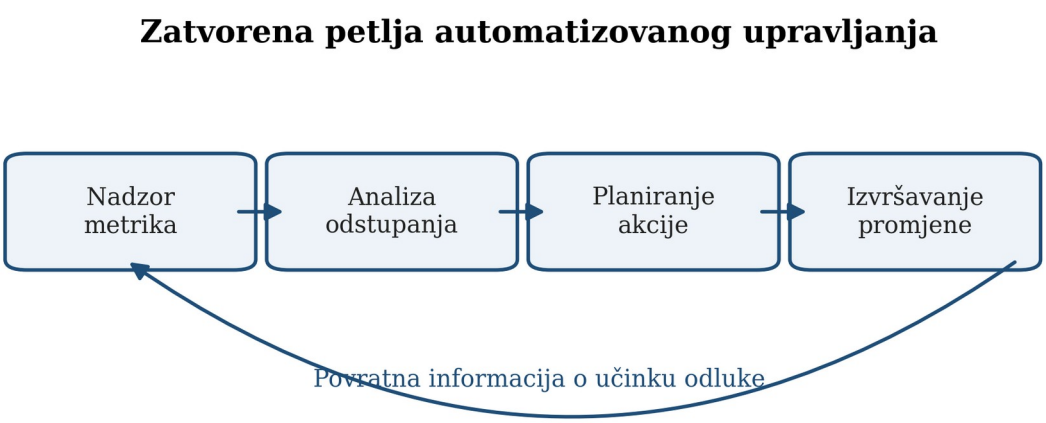
upravljanje resursima predstavlja ključni instrument za postizanje optimalnog odnosa između troškova i kvaliteta usluge.

Važan aspekt upravljanja resursima u oblaku jeste i njegova uloga u očuvanju funkcionalnosti i pouzdanosti sistema. U savremenim cloud okruženjima često se istovremeno izvršava veliki broj heterogenih zadataka, od jednostavnih korisničkih zahtjeva do kompleksnih poslovnih procesa, analitičkih zadataka i servisa sa visokim zahtjevima za dostupnost. Takva složenost zahtijeva mehanizme koji mogu prepoznati prioritete, raspodijeliti resurse prema karakteristikama zadataka i pravovremeno reagovati na promjene u opterećenju. Na taj način upravljanje resursima postaje osnova za održavanje stabilnog i predvidivog rada sistema.

Poseban značaj ovog procesa dolazi do izražaja u cloud okruženjima koja podržavaju zadatke različitog prioriteta i različitih vremenskih ograničenja. Neki zadaci zahtijevaju brzo izvršavanje i strogo poštivanje vremenskih rokova, dok su drugi tolerantniji na kašnjenje. U takvim uslovima upravljanje resursima mora obuhvatiti ne samo pitanje koliko resursa treba dodijeliti određenom zadatku, nego i kada i pod kojim uslovima pristup tim resursima treba biti omogućen. Upravo ta dva pitanja, količina resursa i vrijeme pristupa resursima, čine srž problema planiranja i schedulinga u cloud sistemima.

Savremeni pristupi upravljanju resursima sve više uključuju automatizaciju, prediktivne modele, adaptivne algoritme i dinamičko skaliranje. Takvi mehanizmi omogućavaju da se sistem prilagođava bez stalne ručne intervencije administratora, čime se povećava efikasnost upravljanja i smanjuje vjerovatnoća ljudske greške. Istovremeno, automatizovano upravljanje resursima omogućava bolje iskorištenje infrastrukture, ravnomjerniju raspodjelu opterećenja i kvalitetniji odgovor na promjene u obrascima korištenja usluga [7], [8].

Na osnovu navedenog može se zaključiti da upravljanje resursima u oblaku nije samo tehničko pitanje raspodjele kapaciteta, nego složen proces koji obuhvata performanse, ekonomičnost, pouzdanost, skalabilnost i održivost cloud sistema. Zbog toga ono zauzima centralno mjesto u savremenim istraživanjima iz oblasti cloud computinga i predstavlja jednu od ključnih pretpostavki za kvalitetno funkcionisanje i razvoj cloud infrastrukture.



Slika 3.1. Zatvorena petlja automatizovanog upravljanja resursima

Izvor: izrada autora prema [7] i [8].

3.2. Vrste resursa u cloud okruženju

Da bi se razumjela složenost upravljanja resursima u oblaku, neophodno je prethodno sagledati koje se vrste resursa uopće pojavljuju u cloud okruženju. Cloud infrastruktura obuhvata više kategorija resursa koji se međusobno razlikuju prema funkciji, načinu korištenja i uticaju na ukupne performanse sistema. U najširem smislu, resursi u oblaku mogu se podijeliti na procesorske, memorijske, mrežne i skladišne resurse, pri čemu svaka od tih kategorija ima poseban značaj u procesu alokacije i optimizacije.

3.2.1. Procesorski resursi

Procesorski resursi predstavljaju osnovu izvršavanja zadataka u cloud okruženju. Oni obuhvataju procesorsko vrijeme, broj dostupnih jezgri, kapacitet obrade instrukcija i ukupnu računarsku snagu koja se može dodijeliti korisnicima, aplikacijama i virtualnim mašinama. Alokacija procesorskih resursa od ključnog je značaja za brzinu izvršavanja zadataka, odziv sistema i mogućnost paralelne obrade većeg broja zahtjeva [1].

U cloud okruženju procesorski resursi najčešće se dijele između više korisnika i više virtualnih instanci, zbog čega je neophodno osigurati pravednu i efikasnu raspodjelu. Ako pojedini zadaci dobiju premalo procesorskog vremena, može doći do kašnjenja u izvršavanju i smanjenja performansi. S druge strane, prekomjerno rezervisanje procesorskih kapaciteta može izazvati slabu iskorištenost infrastrukture i povećanje troškova. Upravo zbog toga

upravljanje procesorskim resursima zauzima centralno mjesto u schedulingu i planiranju rada cloud sistema.

3.2.2. Memorijski resursi

Memorijski resursi obuhvataju radnu memoriju potrebnu za izvršavanje aplikacija, obradu podataka i održavanje aktivnih procesa. Količina raspoložive memorije često ima presudan uticaj na stabilnost i efikasnost rada sistema, posebno kod aplikacija koje obrađuju velike količine podataka ili zahtijevaju visok nivo istovremenosti.

U cloud sistemima memorijski resursi se, kao i procesorski, dijele između više virtualnih mašina, kontejnera i aplikacija. Neadekvatna alokacija memorije može izazvati usporavanje rada, prekide u izvršavanju, povećanu potrebu za korištenjem sekundarne memorije i opći pad performansi sistema. Zbog toga upravljanje memorijskim resursima mora osigurati da svaka aplikacija ima dovoljno kapaciteta za stabilan rad, ali i da ukupna memorija bude korištena racionalno i bez nepotrebnih gubitaka.

3.2.3. Mrežni resursi

Mrežni resursi uključuju propusnost mreže, komunikacijske kanale, rutiranje podataka i ukupni kapacitet prenosa informacija između različitih dijelova cloud infrastrukture. U savremenim cloud okruženjima mrežna povezanost ima izuzetno važnu ulogu, jer se veliki broj servisa oslanja na stalnu razmjenu podataka između korisnika, servera, baza podataka i distribuiranih aplikacija.

Upravljanje mrežnim resursima posebno je značajno u sistemima gdje se obrađuju velike količine podataka ili gdje više aplikacija istovremeno koristi iste komunikacijske kapacitete. Nedovoljna mrežna propusnost ili loša raspodjela mrežnih resursa mogu izazvati zagušenja, povećanje latencije i usporavanje rada cijelog sistema. Zbog toga se prilikom upravljanja resursima u oblaku mora voditi računa ne samo o lokalnim računarskim kapacitetima, nego i o sposobnosti mreže da podrži potreban nivo komunikacije.

3.2.4. Skladišni resursi

Skladišni resursi obuhvataju kapacitete za pohranu, organizaciju i pristup podacima u cloud infrastrukturi. Oni uključuju fizičke i virtualne diskovne prostore, baze podataka, distribuirane sisteme za pohranu i različite servise za dugoročno ili privremeno čuvanje podataka. U cloud computingu skladišni resursi imaju izuzetnu važnost, jer veliki broj usluga zavisi od pouzdane, brze i skalabilne pohrane podataka.

Efikasno upravljanje skladišnim resursima zahtijeva balans između kapaciteta, dostupnosti, sigurnosti i brzine pristupa. Ako se skladišni prostor dodjeljuje neadekvatno, može doći do zagušenja, povećanja vremena pristupa podacima ili nepotrebnog rasta troškova. Pored toga, u cloud okruženju posebnu pažnju treba posvetiti replikaciji podataka, rasporedu pohrane i pristupu podacima iz različitih geografskih lokacija. Zbog toga skladišni resursi predstavljaju važnu komponentu ukupnog sistema upravljanja resursima.

Tabela 3.1. Resursi, tipične metrike i posljedice nedovoljne alokacije

Resurs	Tipične metrike	Moguća posljedica
Procesor	Iskorištenost, vrijeme čekanja, throttling	Sporije izvršavanje i rast reda
Memorija	Zauzeće, page faults, OOM događaji	Prekid procesa ili korištenje sporijeg diska
Mreža	Propusnost, latencija, gubitak paketa	Spor prenos i nestabilan odziv
Skladište	IOPS, latencija, kapacitet	Zagušenje i rast vremena pristupa

Izvor: izrada autora prema [1].

3.3. Izazovi upravljanja resursima u oblaku

Upravljanje resursima u cloud okruženju suočava se sa brojnim izazovima koji proizlaze iz same prirode oblaka kao dinamičkog, distribuiranog i višekorisničkog sistema. Za razliku od statičkih informacionih sistema, u kojima su opterećenja često predvidivija i resursi unaprijed dodijeljeni, cloud infrastruktura mora biti sposobna da u realnom vremenu odgovori na promjene broja korisnika, intenziteta zahtjeva i vrsta zadataka koji se izvršavaju. Upravo ta dinamičnost čini upravljanje resursima složenim i zahtjevnim procesom [1].

Jedan od osnovnih izazova jeste promjenjivost opterećenja sistema. U cloud okruženju opterećenje može značajno oscilirati u kratkim vremenskim intervalima, zavisno od ponašanja korisnika, sezonskih promjena, poslovnih procesa ili nepredviđenih događaja. Sistem mora biti sposoban da u takvim okolnostima brzo prilagodi raspoložive resurse, kako bi se izbjeglo i preopterećenje i nepotrebno neiskorišteni kapaciteti.

Drugi važan izazov predstavlja heterogenost zadataka i aplikacija. Cloud okruženja istovremeno podržavaju različite tipove radnih opterećenja, od jednostavnih transakcijskih zahtjeva do složenih analitičkih i računarski intenzivnih procesa. Neki zadaci zahtijevaju nisku latenciju i brzu obradu, dok su drugi više osjetljivi na količinu raspoloživih resursa nego na vrijeme odziva. Upravljanje resursima zbog toga mora uvažiti razlike između zadataka i omogućiti prilagođene strategije alokacije.

Treći izazov ogleda se u potrebi za usklađivanjem performansi i troškova. Cloud sistemi treba da osiguraju visok kvalitet usluge, ali istovremeno moraju poslovati ekonomično. Prekomjerno dodjeljivanje resursa može poboljšati performanse, ali istovremeno povećava troškove korištenja infrastrukture. Nedovoljna alokacija smanjuje troškove na prvi pogled, ali može izazvati pad kvaliteta usluge, nezadovoljstvo korisnika i dodatne indirektne gubitke. Zbog toga je jedan od najvećih izazova pronaći optimalnu ravnotežu između ova dva zahtjeva [1].

Značajan izazov predstavlja i višekorisničko dijeljenje infrastrukture. Više korisnika i više aplikacija često koristi iste fizičke resurse, pa je neophodno osigurati pravednu raspodjelu kapaciteta i spriječiti situacije u kojima pojedini korisnici zauzimaju nesrazmjerno veliki dio raspoloživih resursa. To otvara pitanja prioriteta, kvaliteta usluge, pravednosti raspoređivanja i zaštite interesa različitih korisničkih grupa [10].

Dodatni izazov odnosi se na skalabilnost upravljačkih mehanizama. Kako cloud sistemi rastu, raste i broj virtualnih mašina, kontejnera, korisnika i aplikacija koje treba nadzirati i koordinisati. U takvim uslovima mehanizmi upravljanja resursima moraju biti dovoljno efikasni da i sami ne postanu izvor usporavanja ili dodatnog opterećenja sistema. Drugim riječima, sistem upravljanja mora ostati funkcionalan i kada se infrastruktura i broj zahtjeva značajno povećaju [13], [15].

Važan izazov predstavlja i predvidivost ponašanja sistema. Upravljanje resursima ne odnosi se samo na reakciju na trenutno stanje, nego i na sposobnost procjene budućih potreba. Ako sistem ne može pravovremeno predvidjeti rast opterećenja ili promjenu u obrascima korištenja, postoji veća vjerovatnoća da će resursi biti dodijeljeni prekasno ili neadekvatno. Upravo zbog toga savremeni cloud sistemi sve više koriste adaptivne i prediktivne pristupe u upravljanju resursima [7], [8].

Sve navedeno pokazuje da izazovi upravljanja resursima u oblaku nisu samo tehničke prirode. Oni uključuju i organizacijske, ekonomske i funkcionalne dimenzije, zbog čega upravljanje

resursima predstavlja jedno od najsloženijih i najvažnijih pitanja savremenog cloud computinga.

3.4. Odnos između performansi, funkcionalnosti i cijene

Jedna od ključnih karakteristika upravljanja resursima u oblaku jeste stalna potreba za uspostavljanjem ravnoteže između performansi sistema, njegove funkcionalnosti i cijene pružanja usluge. Ova tri elementa međusobno su povezana i čine osnovne kriterije na osnovu kojih se procjenjuje kvalitet i efikasnost cloud sistema. Svaka odluka o alokaciji resursa direktno ili indirektno utiče na sve tri dimenzije, zbog čega se upravljanje resursima može posmatrati kao proces optimizacije njihovog međusobnog odnosa [1].

Performanse se odnose na sposobnost sistema da brzo i efikasno obradi korisničke zahtjeve, održi stabilnost rada i obezbijedi prihvatljivo vrijeme odziva. U cloud okruženju visoke performanse obično zahtijevaju dovoljnu količinu procesorskih, memorijskih, mrežnih i skladišnih resursa, kao i kvalitetne mehanizme schedulinga i upravljanja opterećenjem. Međutim, povećanje performansi najčešće podrazumijeva i angažovanje većeg broja resursa, što direktno utiče na troškove rada sistema.

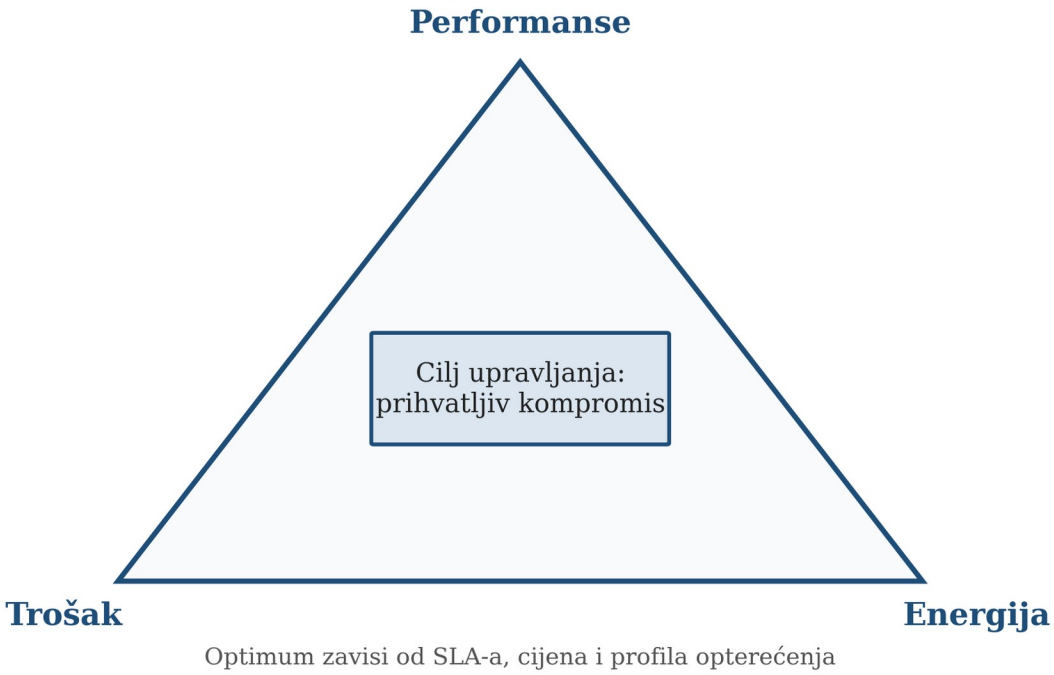
Funkcionalnost sistema odnosi se na mogućnost izvršavanja željenih zadataka, podršku različitim vrstama aplikacija i zadovoljavanje korisničkih potreba. U određenim situacijama funkcionalnost može zavisiti od dostupnosti dodatnih resursa ili od sposobnosti sistema da istovremeno podrži više servisa i korisnika. Ako resursi nisu pravilno raspoređeni, pojedine funkcije mogu biti ograničene, usporene ili čak privremeno nedostupne. Na taj način loše upravljanje resursima može indirektno smanjiti funkcionalnu vrijednost sistema.

Cijena predstavlja ekonomsku dimenziju korištenja cloud resursa i obuhvata troškove infrastrukture, rada sistema, prenosa podataka, pohrane, održavanja i potrošnje energije. Jedna od osnovnih prednosti cloud modela jeste mogućnost racionalnijeg korištenja resursa i plaćanja prema stvarnoj upotrebi. Međutim, ako upravljanje resursima nije efikasno, troškovi mogu značajno porasti usljed nepotrebnog rezervisanja kapaciteta, neravnomjerne raspodjele opterećenja ili prevelike potrošnje energije.

Između performansi, funkcionalnosti i cijene postoji stalna međuzavisnost. Povećanje performansi često zahtijeva dodatne resurse i povećava cijenu, dok smanjenje troškova može dovesti do ograničenja u performansama ili funkcionalnosti ako se resursi smanje ispod optimalnog nivoa. Zbog toga cilj upravljanja resursima nije jednostavno maksimizirati jedan

od ovih kriterija, nego pronaći ravnotežu koja omogućava zadovoljavajući nivo performansi i funkcionalnosti uz prihvatljive troškove.

Upravo u toj ravnoteži leži suština savremenog cloud resource managementa. Efikasni mehanizmi upravljanja resursima nastoje obezbijediti da se resursi dodjeljuju tamo gdje donose najveću korist, da se izbjegne i podkapacitiranost i prekomjerno rezervisanje, te da se sistem održava u stanju optimalne iskorištenosti. Time upravljanje resursima direktno utiče na kvalitet usluge i održivost cloud poslovnih modela.



Slika 3.2. Višekriterijska ravnoteža upravljanja resursima

Izvor: izrada autora.

3.5. Energetska efikasnost u cloud okruženju

Pored performansi i troškova, savremena istraživanja iz oblasti cloud computinga sve veći značaj pridaju energetskoj efikasnosti. Razlog za to leži u činjenici da veliki podatkovni centri i cloud infrastrukture troše značajne količine električne energije, kako za rad servera i mrežne opreme, tako i za hlađenje, održavanje i prateće infrastrukturne sisteme. Zbog toga je upravljanje resursima u oblaku sve više povezano ne samo sa tehničkom i ekonomskom, nego i sa energetskom optimizacijom sistema [17].

Energetska efikasnost u cloud okruženju podrazumijeva sposobnost sistema da izvrši što veći obim korisnog rada uz što manju potrošnju energije. Ovaj cilj ne ostvaruje se samo

korištenjem energetski efikasnijeg hardvera, nego i pravilnim upravljanjem resursima, odnosno inteligentnom alokacijom zadataka, gašenjem ili konsolidacijom nedovoljno iskorištenih instanci, optimizacijom raspoređivanja i dinamičkim prilagođavanjem kapaciteta stvarnim potrebama.

Neefikasno upravljanje resursima može dovesti do situacije u kojoj veliki broj servera radi sa niskim stepenom iskorištenosti, ali i dalje troši značajne količine energije. U takvim slučajevima troškovi rada rastu, a istovremeno se povećava negativan uticaj na okolinu. Nasuprot tome, pažljivo raspoređivanje zadataka i konsolidacija radnih opterećenja omogućavaju da se određeni dijelovi infrastrukture privremeno rasterete ili isključe, čime se ostvaruju uštede energije bez značajnog narušavanja kvaliteta usluge [6].

Energetska efikasnost usko je povezana i sa dinamičkim skaliranjem. Kada sistem može automatski povećavati ili smanjivati broj aktivnih resursa u skladu sa trenutnim opterećenjem, smanjuje se potreba da veliki kapaciteti budu stalno aktivni bez stvarne potrebe. Takav pristup omogućava ekonomičniji i održiviji rad cloud infrastrukture, posebno u okruženjima gdje opterećenje značajno varira tokom dana ili tokom određenih poslovnih ciklusa [6], [7].

Pored ekonomskih koristi, energetska efikasnost ima i širi društveni i strateški značaj. Sve veći broj organizacija nastoji uskladiti poslovanje sa principima održivog razvoja, smanjiti ugljični otisak i povećati ekološku odgovornost svojih informacionih sistema. Zbog toga je upravljanje resursima koje doprinosi manjoj potrošnji energije danas jedan od važnih kriterija kvaliteta cloud infrastrukture.

Na osnovu navedenog može se zaključiti da energetska efikasnost nije sporedna, već sastavna komponenta savremenog upravljanja resursima u oblaku. Efikasna alokacija resursa, odgovarajući mehanizmi schedulinga i dinamičko skaliranje ne doprinose samo boljoj iskorištenosti i manjim troškovima, nego i održivijem radu cijelog cloud sistema [6], [17].

4. Alokacija resursa i scheduling mehanizmi u oblaku

4.1. Pojam alokacije resursa u cloud okruženju

Alokacija resursa u cloud okruženju predstavlja proces dodjeljivanja raspoloživih infrastrukturnih kapaciteta korisnicima, aplikacijama, virtualnim mašinama, kontejnerima i pojedinačnim zadacima, u skladu sa zahtjevima sistema i postavljenim pravilima upravljanja. U najširem smislu, alokacija resursa odgovara na dva osnovna pitanja: koliko resursa treba dodijeliti određenom zahtjevu i u kojem trenutku mu treba omogućiti pristup tim resursima. Upravo ta dva pitanja čine suštinu upravljanja cloud sistemima, jer od njihove pravilne procjene zavise performanse, cijena i ukupna efikasnost infrastrukture [1].

Za razliku od tradicionalnih informacionih sistema, u kojima se resursi često dodjeljuju unaprijed i relativno statično, u cloud okruženju alokacija mora biti dinamična i prilagodljiva. Razlog za to leži u činjenici da cloud sistemi rade u uslovima promjenjivog opterećenja, različitih prioriteta korisničkih zahtjeva i heterogenih vrsta zadataka. Neki zahtjevi su kratkotrajni i laki za obradu, dok drugi mogu biti dugotrajni, računarski intenzivni i osjetljivi na vrijeme izvršenja. Zbog toga alokacija resursa u oblaku ne predstavlja samo tehničku dodjelu kapaciteta, nego i složen upravljački proces koji mora uzeti u obzir performanse, pravičnost, rokove izvršavanja i ekonomičnost korištenja resursa.

Učinkovita alokacija resursa podrazumijeva da se resursi ne dodjeljuju ni previše ni premalo. Ako sistem dodijeli više resursa nego što je stvarno potrebno, dolazi do neefikasnog korištenja infrastrukture i povećanja troškova. Ako dodijeli manje resursa nego što zahtjev traži, rezultat može biti kašnjenje, zagušenje, pad kvaliteta usluge ili neispunjavanje vremenskih ograničenja. Zbog toga je cilj savremenih mehanizama alokacije pronaći optimalnu raspodjelu koja balansira zahtjeve korisnika sa ograničenjima sistema.

U kontekstu cloud computinga alokacija resursa usko je povezana sa pojmom schedulinga, odnosno planiranja i raspoređivanja izvršavanja zadataka. Alokacija određuje koji će resursi biti dodijeljeni, dok scheduling određuje kojim redoslijedom, u kojem vremenu i pod kojim pravilima će zadaci pristupiti tim resursima. Tek njihovo zajedničko posmatranje omogućava potpuno razumijevanje načina na koji cloud sistemi funkcionišu i na koji način mogu biti optimizirani.

4.2. Politike upravljanja resursima

Politike upravljanja resursima predstavljaju skup pravila i principa na osnovu kojih cloud sistem donosi odluke o raspodjeli raspoloživih kapaciteta. One definišu prioritete sistema, uslove dodjele resursa, kriterije za raspoređivanje zadataka i način reagovanja na promjene u opterećenju. Drugim riječima, politike upravljanja resursima određuju logiku ponašanja sistema u situacijama kada više korisnika ili aplikacija istovremeno traži pristup ograničenim resursima [1], [18].

Jedan od osnovnih zadataka politika upravljanja jeste usklađivanje međusobno suprotstavljenih ciljeva. Cloud sistem istovremeno nastoji postići visoke performanse, zadovoljiti korisničke zahtjeve, osigurati pravičnost u raspodjeli kapaciteta, smanjiti troškove i racionalno koristiti energiju. Budući da se svi ovi ciljevi ne mogu uvijek ostvariti u istoj mjeri, politike upravljanja služe da definišu koje će kriterije sistem u određenim okolnostima staviti u prvi plan.

Politike mogu biti zasnovane na različitim principima. Neke daju prednost zadacima višeg prioriteta, neke nastoje osigurati ravnomjernu raspodjelu među korisnicima, neke su usmjerene na ispunjavanje vremenskih rokova, dok druge favorizuju najveću moguću iskorištenost infrastrukture. Također, određene politike mogu biti fokusirane na minimizaciju troškova, dok druge prioritet daju očuvanju kvaliteta usluge ili energetskoj efikasnosti.

Važna podjela politika upravljanja resursima odnosi se na statičke i dinamičke politike. Statičke politike oslanjaju se na unaprijed definisana pravila i manje su prilagodljive promjenama u radnom opterećenju. Dinamičke politike, nasuprot tome, prilagođavaju svoje odluke trenutnom stanju sistema, obrascima korištenja i procjeni budućih zahtjeva. U savremenim cloud okruženjima dinamičke politike imaju posebnu važnost, jer omogućavaju fleksibilnije i efikasnije upravljanje resursima.

Kvalitet politike upravljanja resursima ogleda se u njenoj sposobnosti da donese uravnotežene odluke u složenim i promjenjivim uslovima. Dobro postavljena politika upravljanja ne znači samo da će sistem “raditi”, nego da će raditi stabilno, predvidivo i uz optimalno korištenje dostupnih kapaciteta.

4.3. Planiranje i raspoređivanje zadataka u oblaku

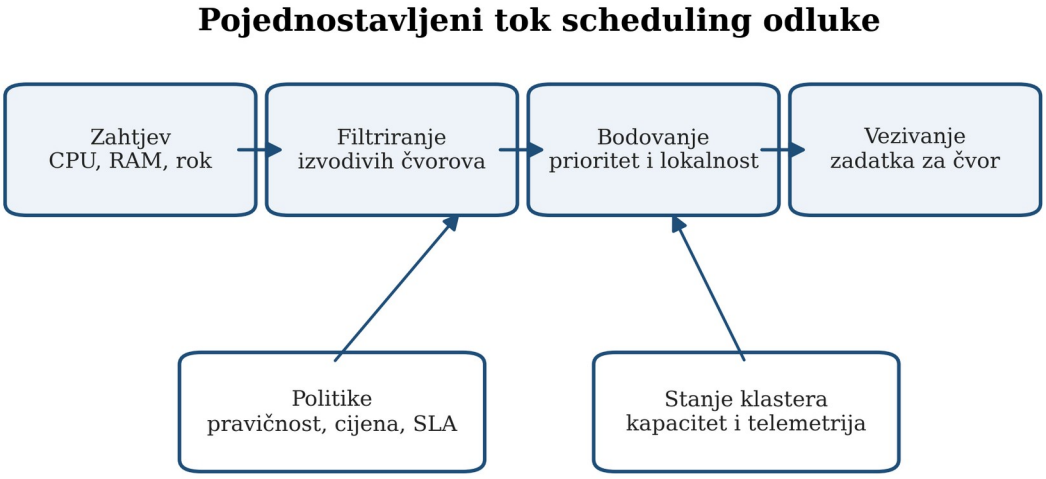
Planiranje i raspoređivanje zadataka, odnosno scheduling, predstavlja ključni operativni mehanizam pomoću kojeg cloud sistem odlučuje kojim redoslijedom će pojedini zadaci biti

izvršeni i kada će dobiti pristup raspoloživim resursima. Scheduling je naročito važan u situacijama kada više zahtjeva istovremeno konkurira za iste resurse, što je tipično za cloud okruženja zasnovana na dijeljenju infrastrukture [1], [18].

Osnovna svrha schedulinga jeste uspostavljanje reda izvršavanja koji će omogućiti efikasan rad sistema. Međutim, pojam efikasnosti u ovom kontekstu nije jednoznačan. U nekim situacijama efikasan scheduling znači minimizaciju prosječnog vremena čekanja, u drugim maksimiziranje protoka sistema, u trećim ispunjavanje rokova izvršavanja, a u četvrtim osiguravanje pravične raspodjele među korisnicima. Upravo zbog toga ne postoji jedan univerzalno najbolji scheduling mehanizam, nego izbor pristupa zavisi od ciljeva i karakteristika konkretnog sistema.

U cloud sistemima scheduling ima dodatnu složenost zbog višeslojnosti infrastrukture. Odluke o raspoređivanju ne odnose se samo na redoslijed izvršavanja zadataka, nego i na izbor fizičke ili virtualne lokacije izvršavanja, raspodjelu kapaciteta između virtualnih mašina, usklađivanje rada mrežnih i skladišnih resursa te reagovanje na promjene u opterećenju. Zbog toga scheduling u oblaku nije izolovan proces, nego sastavni dio šireg sistema upravljanja resursima.

Pravilno organizovan scheduling omogućava da cloud sistem istovremeno obradi veći broj zahtjeva, skrati vrijeme zadržavanja zadataka u redu čekanja i poveća iskorištenost raspoloživih resursa. Istovremeno, loše postavljen scheduling može dovesti do zastoja, neefikasne raspodjele, gladi pojedinih procesa i neispunjavanja zahtjeva korisnika. U tom smislu, scheduling je jedan od najvažnijih instrumenata za ostvarivanje ključnih ciljeva cloud resource managementa.



Slika 4.1. Pojednostavljeni tok scheduling odluke

Izvor: izrada autora prema [13], [15] i [21].

4.4. Preemptivno i nepreemptivno planiranje

Jedna od temeljnih podjela scheduling pristupa odnosi se na preemptivno i nepreemptivno planiranje. Ova podjela zasniva se na pitanju da li sistem može prekinuti izvršavanje jednog zadatka kako bi oslobodio resurse za drugi zadatak koji ima viši prioritet ili strožije vremenske zahtjeve [1].

Preemptivno planiranje omogućava da se zadatak koji je trenutno u izvršavanju privremeno zaustavi, a njegovi resursi dodijele drugom zadatku. Ovakav pristup posebno je važan u sistemima gdje postoje hitni zahtjevi, zadaci sa visokim prioritetom ili procesi koji moraju biti izvršeni unutar strogih vremenskih ograničenja. Prednost preemptivnog pristupa jeste u tome što povećava fleksibilnost sistema i omogućava brže reagovanje na promjene u opterećenju i prioritetima.

Međutim, preemptivno planiranje ima i određene nedostatke. Svako prekidanje zadatka može proizvesti dodatni upravljački trošak, povećati složenost sistema i uzrokovati gubitak dijela već uloženog procesorskog vremena. Ako se preuzimanje resursa dešava prečesto, sistem može postati manje stabilan, a ukupna efikasnost može pasti usljed čestih promjena konteksta i dodatnih troškova koordinacije.

Nepreemptivno planiranje zasniva se na principu da zadatak, jednom kada dobije resurse i započne izvršavanje, zadržava ih do završetka ili do trenutka kada sam oslobodi kapacitete.

Ovakav pristup je jednostavniji za implementaciju i često pogodniji za okruženja gdje je važno očuvati kontinuitet izvršavanja i smanjiti upravljačku složenost. S druge strane, nepreemptivni pristup može dovesti do toga da zadaci višeg prioriteta nepotrebno čekaju, posebno ako sistem trenutno izvršava dugotrajan zadatak nižeg prioriteta.

U cloud okruženju izbor između preemptivnog i nepreemptivnog planiranja zavisi od prirode radnog opterećenja, ciljeva sistema i zahtjeva korisnika. Sistemi koji podržavaju aplikacije osjetljive na vrijeme odziva češće koriste elemente preemptivnog planiranja, dok se u nekim drugim slučajevima preferiraju jednostavniji nepreemptivni modeli. U praksi se često koriste i kombinovani pristupi koji nastoje spojiti prednosti oba modela.

4.5. Prioritetno planiranje i raspoređivanje

Prioritetno planiranje predstavlja scheduling pristup u kojem se zadacima dodjeljuju različiti nivoi važnosti, a odluke o raspoređivanju zasnivaju se na tim prioritetima. U cloud okruženju ovaj pristup ima posebno veliki značaj, jer svi korisnički zahtjevi nemaju jednaku poslovnu, funkcionalnu ili vremensku vrijednost. Neki zadaci mogu biti kritični za rad sistema ili vezani za stroge ugovorne obaveze, dok drugi imaju niži nivo hitnosti [1].

Kod prioritetnog planiranja sistem nastoji prvo izvršiti zadatke višeg prioriteta, a tek potom one manje važne. Ovakav pristup može značajno poboljšati odziv sistema za kritične aplikacije i omogućiti bolje upravljanje kvalitetom usluge. Posebno je koristan u cloud sistemima koji podržavaju više klasa korisnika ili više nivoa usluge, gdje nije dovoljno sve zahtjeve tretirati potpuno jednako.

Ipak, prioritetno planiranje nosi i određene rizike. Ako se prioriteti postave rigidno i bez dodatnih korektivnih mehanizama, može doći do pojave gladi, odnosno situacije u kojoj zadaci nižeg prioriteta predugo čekaju ili se praktično nikada ne izvrše. Zbog toga se u savremenim sistemima često koriste tehnike koje koriguju prioritet tokom vremena, vode računa o dužini čekanja ili kombinuju prioritetne kriterije sa principima pravičnosti.

U cloud okruženju prioriteti se mogu određivati na osnovu različitih parametara: vrste korisnika, klase usluge, vremena pristizanja zahtjeva, procijenjenog trajanja zadatka, poslovne važnosti aplikacije ili zahtijevanog nivoa kvaliteta usluge. Time prioritetno planiranje postaje važan alat za prilagođavanje rada sistema konkretnim ciljevima organizacije i očekivanjima korisnika.

4.6. Fair queuing i deadline scheduling

U savremenim cloud sistemima značajnu ulogu imaju scheduling pristupi koji nastoje postići pravičnost u raspodjeli resursa ili osigurati izvršavanje zadataka unutar definisanih vremenskih rokova. Dva posebno važna koncepta u tom smislu jesu fair queuing i deadline scheduling, koji se i u literaturi o resource managementu navode kao važni mehanizmi raspoređivanja [10], [11].

Fair queuing polazi od ideje da korisnici ili tokovi zadataka treba da imaju pravedan pristup raspoloživim resursima. Cilj ovog pristupa nije nužno dati apsolutnu prednost najhitnijim ili najvećim zadacima, nego spriječiti da pojedini korisnici zauzmu nesrazmjerno veliki dio sistema na štetu drugih. U cloud okruženju, gdje više korisnika dijeli istu infrastrukturu, ovakav pristup je izuzetno značajan za očuvanje ravnoteže, predvidivosti i zadovoljstva korisnika. Fairness mehanizmi posebno dolaze do izražaja kada sistem mora istovremeno služiti različitim aplikacijama i različitim klasama opterećenja [10].

Deadline scheduling zasniva se na vremenskim rokovima unutar kojih zadaci moraju biti izvršeni. U ovom pristupu redoslijed izvršavanja ne određuje se primarno prema prioritetu korisnika ili vremenu dolaska zadatka, nego prema tome koliko je zadatak blizu svom krajnjem roku. Ovaj pristup je naročito važan u okruženjima gdje kašnjenje izvršavanja može imati ozbiljne posljedice po kvalitet usluge, ugovorne obaveze ili samu funkcionalnost sistema [9], [18].

Prednost deadline pristupa jeste u tome što omogućava sistemu da se fokusira na zadatke čije bi kašnjenje imalo najveći negativan uticaj. Međutim, ako se koristi bez dodatnih korekcija, takav pristup može zanemariti zadatke koji nemaju stroge rokove, iako su također važni za ukupni rad sistema. Zbog toga se u praksi često kombinuju fairness i deadline elementi, kako bi se istovremeno osigurala i pravičnost i vremenska efikasnost.

4.7. Algoritmi i modeli alokacije resursa

Algoritmi i modeli alokacije resursa predstavljaju operativnu osnovu pomoću koje cloud sistem sprovodi politike upravljanja i scheduling odluke. Njihova funkcija nije samo da “dodijele” resurse, nego da to učine na način koji odgovara strateškim ciljevima sistema, kao što su maksimiziranje protoka, minimiziranje vremena obrta, osiguravanje pravičnosti, smanjenje troškova ili povećanje energetske efikasnosti [18].

U zavisnosti od složenosti sistema i prirode zahtjeva, algoritmi alokacije mogu biti jednostavni ili veoma sofisticirani. Jednostavniji modeli zasnivaju se na unaprijed definisanim pravilima i lokalnim odlukama, dok napredniji pristupi uključuju procjenu opterećenja, predikciju budućih zahtjeva, višekriterijsko odlučivanje i koordinaciju između više upravljačkih slojeva. U literaturi o cloud resource managementu poseban značaj imaju modeli zasnovani na kontroli sistema, pravičnosti, vremenskim ograničenjima i korisnosti. Dostupni sadržajni opisi poglavlja koje koristiš upravo navode kontrolnu teoriju, fairness pristupe i upravljanje resursima kao centralne teme.

Važan aspekt algoritama alokacije jeste njihova sposobnost da djeluju u uslovima nesigurnosti. Cloud sistem često ne raspolaže potpunom informacijom o budućem opterećenju, dužini izvršavanja zadataka ili promjenama korisničkog ponašanja. Zbog toga algoritmi moraju biti dovoljno adaptivni da mogu korigovati odluke kada se stanje sistema promijeni. Upravo ta adaptivnost razlikuje savremene cloud algoritme od rigidnih pristupa karakterističnih za tradicionalna okruženja [15].

Modeli alokacije resursa također moraju voditi računa o tome da se odluke donose na više nivoa. Nije dovoljno odrediti samo koji korisnik dobija koliko procesorskog vremena; potrebno je uskladiti i rad virtualnih mašina, mrežnih tokova, skladišnih kapaciteta i mehanizama za balansiranje opterećenja. Time alokacija resursa postaje višeslojni proces u kojem odluka na jednom nivou može imati direktne posljedice na performanse drugog nivoa sistema [13], [14].

Tabela 4.1. Komparativni pregled scheduling pristupa

Pristup	Primarni cilj	Prednost	Ograničenje
Preemptivni	Brz odziv važnih zadataka	Oslobađa resurse za hitne poslove	Trošak prekida i nastavka
Prioritetni	Zaštita poslovno važnih klasa	Jednostavna kontrola važnosti	Rizik gladi nižih prioriteta
Pravični	Ravnomjerna raspodjela	Ograničava monopolizaciju	Ne optimizira nužno rok
Deadline	Završetak prije roka	Usmjeren na vremenski cilj	Može potisnuti zadatke bez roka

Pristup	Primarni cilj	Prednost	Ograničenje
Data-aware	Lokalnost podataka	Smanjuje mrežni prenos	Ograničen rasporedom podataka

Izvor: izrada autora prema [9]-[12] i [18].

4.8. Upravljanje resursima kao osnova optimizacije cloud sistema

Na osnovu prethodno razmotrenih pojmova može se zaključiti da su alokacija resursa i scheduling mehanizmi suštinska osnova optimizacije cloud sistema. Cloud infrastruktura ne postiže efikasnost sama po sebi samo zato što raspolaže velikim brojem servera ili velikim kapacitetima. Stvarna vrijednost sistema zavisi od načina na koji se ti kapaciteti raspoređuju, usklađuju i prilagođavaju promjenjivim potrebama korisnika i aplikacija [1].

Efikasno upravljanje resursima omogućava da cloud sistem obradi veći broj zahtjeva u jedinici vremena, skrati vrijeme čekanja i izvršavanja zadataka, poveća iskorištenost infrastrukture i smanji operativne troškove. Osim toga, dobro postavljeni scheduling i alokacijski mehanizmi doprinose stabilnijem radu sistema, ravnomjernijoj raspodjeli opterećenja i većoj predvidivosti kvaliteta usluge [13].

Posebna vrijednost upravljanja resursima ogleda se u tome što ono povezuje tehničku, ekonomsku i funkcionalnu dimenziju cloud computinga. Pravilna alokacija resursa ne znači samo bolji tehnički odziv sistema, nego i racionalnije korištenje infrastrukture, smanjenje nepotrebnih troškova i stvaranje uslova za širenje funkcionalnosti bez nesrazmjernog povećanja opterećenja. Na taj način resource management postaje centralni mehanizam pomoću kojeg cloud sistemi ostvaruju svoju osnovnu prednost: fleksibilno i efikasno pružanje usluga u promjenjivom okruženju.

Upravo zbog toga se alokacija resursa i scheduling mehanizmi mogu posmatrati kao jezgro upravljanja cloud infrastrukturom. Njihova kvaliteta direktno utiče na to da li će sistem biti samo tehnički operativan ili zaista optimizovan u pogledu performansi, troškova, pravičnosti i održivosti. Iz tog razloga ova oblast zauzima centralno mjesto u teorijskim i praktičnim istraživanjima cloud computinga.

5. Dinamičko skaliranje i optimizacija resursa u oblaku

Dinamička priroda cloud okruženja zahtijeva primjenu mehanizama koji omogućavaju kontinuirano prilagođavanje raspoloživih resursa stvarnim potrebama korisnika i aplikacija. Za razliku od tradicionalnih računarskih sistema, kod kojih se kapacitet infrastrukture najčešće unaprijed planira i ostaje relativno nepromijenjen tokom dužeg vremenskog perioda, cloud sistemi omogućavaju povećavanje ili smanjivanje resursa u skladu sa trenutnim opterećenjem. Ova karakteristika predstavlja jednu od osnovnih prednosti računarstva u oblaku, ali istovremeno postavlja složene zahtjeve pred sisteme za upravljanje resursima [7], [8].

Opterećenje cloud aplikacija može se značajno mijenjati tokom vremena. Promjene mogu biti postepene i predvidive, ali mogu nastati i iznenada, kao posljedica neočekivanog povećanja broja korisnika, poslovnih aktivnosti, sezonskih događaja, kvarova ili drugih vanjskih okolnosti. U takvim uslovima statička alokacija resursa nije dovoljno efikasna, jer unaprijed rezervisani kapaciteti mogu biti nedovoljni tokom perioda visokog opterećenja, dok u periodima manjeg opterećenja mogu ostati neiskorišteni.

Dinamičko skaliranje omogućava cloud sistemu da reaguje na promjene opterećenja povećavanjem ili smanjivanjem količine dodijeljenih resursa. Na taj način nastoji se održati zadovoljavajući nivo performansi, izbjeći preopterećenje sistema i spriječiti nepotrebno korištenje infrastrukture. Skaliranje je usko povezano sa procesima nadzora, alokacije, raspoređivanja zadataka i balansiranja opterećenja, zbog čega predstavlja važan dio ukupnog sistema za upravljanje resursima [7].

Optimizacija resursa obuhvata skup metoda i postupaka kojima se nastoji postići najbolji mogući odnos između performansi sistema, kvaliteta usluge, troškova i potrošnje energije. U cloud okruženju optimizacija se ne odnosi samo na povećanje brzine izvršavanja aplikacija, nego i na racionalno korištenje procesorskih, memorijskih, mrežnih i skladišnih kapaciteta. Kvalitetna optimizacija mora uzeti u obzir različite vrste aplikacija, njihove prioritete, ugovorene nivoe usluge i promjenjivu dostupnost infrastrukture.

Poseban izazov predstavlja činjenica da se pojedinačni ciljevi optimizacije često međusobno razlikuju ili su u određenoj mjeri suprotstavljeni. Povećanje broja aktivnih servera i virtualnih mašina može poboljšati vrijeme odziva aplikacije, ali istovremeno povećava troškove korištenja infrastrukture i potrošnju električne energije. Smanjenje broja aktivnih resursa

može doprinijeti nižim troškovima, ali može dovesti do preopterećenja, smanjenja performansi i narušavanja kvaliteta usluge. Zbog toga se odluke o skaliranju ne mogu zasnivati samo na jednom kriteriju, nego moraju predstavljati rezultat razmatranja više tehničkih i ekonomskih faktora.

5.1. Pojam i značaj dinamičkog skaliranja

Dinamičko skaliranje predstavlja proces prilagođavanja količine računarskih resursa trenutnim zahtjevima aplikacije ili sistema. Ono podrazumijeva mogućnost dodavanja resursa kada se opterećenje poveća, kao i njihovog smanjivanja ili oslobađanja kada se potreba za njima smanji. Cilj dinamičkog skaliranja jeste osigurati da aplikacija u svakom trenutku raspolaže dovoljnim kapacitetima za izvršavanje zahtjeva, bez nepotrebnog rezervisanja i plaćanja resursa koji se ne koriste [7], [8].

Potreba za dinamičkim skaliranjem proizlazi iz promjenjive prirode opterećenja cloud aplikacija. Zahtjevi za resursima mogu zavisiti od vremena u toku dana, dana u sedmici, sezonskih događaja, ponašanja korisnika i različitih nepredvidivih okolnosti. Opterećenje može postepeno rasti ili opadati, ali se mogu pojaviti i nagli vrhovi koje je teško unaprijed predvidjeti. Nova web usluga, na primjer, u početku može imati mali broj korisničkih zahtjeva, a zatim, ukoliko postane uspješna, doživjeti veoma brz rast opterećenja. Slično tome, određeni sistemi mogu biti izloženi velikom broju zahtjeva samo u određenim vremenskim periodima ili nakon pojave posebnih događaja.

Kod statičkog načina upravljanja unaprijed se određuje količina resursa koja će biti dostupna aplikaciji. Takav pristup može biti prihvatljiv kada je opterećenje relativno stabilno i predvidivo. Međutim, u slučaju velikih promjena zahtjeva statička alokacija može dovesti do nedovoljne ili prekomjerne dodjele resursa. Nedovoljna alokacija nastaje kada dodijeljeni kapaciteti ne mogu obraditi postojeće opterećenje, dok prekomjerna alokacija nastaje kada je aplikaciji dodijeljeno više resursa nego što joj je stvarno potrebno.

Nedovoljna alokacija resursa može izazvati povećanje vremena odziva, stvaranje redova čekanja, smanjenje protoka sistema i privremenu nedostupnost pojedinih usluga. Ukoliko se aplikacija izvršava u skladu sa ugovorom o nivou usluge, smanjenje performansi može predstavljati i kršenje definisanih obaveza pružaoca cloud usluge. Posljedice mogu obuhvatiti nezadovoljstvo korisnika, finansijske kazne, gubitak prihoda i smanjenje povjerenja u uslugu.

Prekomjerna alokacija, s druge strane, osigurava postojanje dodatnog kapaciteta, ali dovodi do slabije iskorištenosti infrastrukture. Resursi koji su rezervisani, a ne izvršavaju korisne

zadatke, i dalje mogu stvarati troškove i trošiti energiju. Dugoročno oslanjanje na prekomjerno obezbjeđivanje kapaciteta nije ekonomski opravdano, posebno u velikim podatkovnim centrima sa velikim brojem fizičkih servera i virtualnih mašina.

Dinamičko skaliranje nastoji izbjeći oba navedena problema. Umjesto određivanja stalne količine kapaciteta, sistem prati stanje aplikacije i infrastrukture, nakon čega donosi odluke o povećavanju ili smanjivanju resursa. Kada se utvrdi da postojeći kapaciteti više nisu dovoljni, aplikaciji se mogu dodijeliti dodatne virtualne mašine, procesorsko vrijeme, memorija ili drugi resursi. Kada opterećenje opadne, višak kapaciteta može se osloboditi i staviti na raspolaganje drugim korisnicima i aplikacijama.

Osnovu dinamičkog skaliranja predstavlja elastičnost cloud okruženja. Elastičnost označava sposobnost sistema da brzo obezbijedi dodatne resurse i da ih, nakon prestanka potrebe, ponovo oslobodi. Sa stanovišta korisnika, dostupni kapaciteti mogu ostavljati utisak gotovo neograničenog skupa resursa koji se može koristiti prema potrebi. Ipak, fizička infrastruktura svakog cloud sistema ima ograničen kapacitet, zbog čega pružalac usluge mora istovremeno upravljati zahtjevima velikog broja aplikacija i korisnika [2], [8].

Značaj elastičnosti dodatno se ogleda u modelu naplate prema stvarnoj upotrebi. Korisnik ne mora unaprijed kupovati i održavati infrastrukturu projektovanu za najveće moguće opterećenje, nego može koristiti dodatne kapacitete samo u periodima kada su oni potrebni. Dinamičko skaliranje tako omogućava neposrednije povezivanje troškova sa stvarnim intenzitetom korištenja aplikacije. Time se smanjuju početna ulaganja i rizik od dugotrajnog posjedovanja nedovoljno iskorištene infrastrukture.

Za uspješnu primjenu dinamičkog skaliranja potrebno je postojanje raspoloživog skupa resursa koji se mogu dodjeljivati i oslobađati na zahtjev. Pored toga, neophodan je sistem za nadzor koji prikuplja podatke o stanju virtualnih mašina, servera i aplikacija. Sistem može pratiti iskorištenost procesora, zauzeće memorije, mrežni saobraćaj, broj primljenih zahtjeva, dužinu reda čekanja, vrijeme odziva i druge pokazatelje relevantne za donošenje odluka.

Na osnovu prikupljenih podataka upravljački mehanizam određuje da li je potrebno izvršiti skaliranje. Odluka se može donijeti na osnovu unaprijed definisanih pragova. Na primjer, ukoliko iskorištenost procesora tokom određenog vremenskog intervala pređe zadanu vrijednost, sistem može pokrenuti dodatnu virtualnu mašinu. Ukoliko iskorištenost duže vrijeme ostane ispod donjeg praga, jedna ili više instanci mogu biti zaustavljene. Korištenje

gornjeg i donjeg praga smanjuje vjerovatnoću prečestog dodavanja i uklanjanja resursa usljed kratkotrajnih promjena opterećenja [7], [20].

Dinamičko skaliranje može biti reaktivno ili proaktivno. Reaktivno skaliranje pokreće se nakon što sistem utvrdi da je došlo do promjene određenog pokazatelja. Prednost ovog pristupa ogleda se u relativnoj jednostavnosti, jer se odluke donose na osnovu stvarno izmjerenog stanja. Njegov nedostatak jeste mogućnost kašnjenja, pošto je potrebno određeno vrijeme za otkrivanje promjene, donošenje odluke i aktiviranje dodatnih resursa [7], [8].

Proaktivno skaliranje zasniva se na predviđanju budućeg opterećenja. Analizom historijskih podataka i uočenih obrazaca sistem može unaprijed pripremiti potrebne resurse. Takav pristup je koristan kada se promjene opterećenja ponavljaju u pravilnim vremenskim intervalima. Međutim, kvalitet proaktivnog skaliranja zavisi od tačnosti predviđanja, dok neočekivani događaji i nagli skokovi opterećenja mogu zahtijevati dodatnu reaktivnu intervenciju [7], [8].

Važan faktor predstavlja vrijeme potrebno za aktiviranje resursa. Pokretanje nove virtualne mašine, učitavanje operativnog sistema, pokretanje aplikacije i uključivanje nove instance u sistem za balansiranje opterećenja ne izvršavaju se trenutno. Ako postupak traje duže, odluka o skaliranju mora se donijeti dovoljno rano kako bi novi kapacitet postao dostupan prije nego što dođe do značajnog pada performansi. Slično tome, prerano uklanjanje resursa može dovesti do situacije u kojoj preostali kapaciteti ne mogu zadovoljiti naknadno povećanje opterećenja [7].

Dinamičko skaliranje dodatno je otežano mogućnošću kvarova i premještanja aktivnih aplikacija. Sistem za upravljanje resursima ne mora reagovati samo na vanjske promjene opterećenja, nego i na unutrašnje događaje, kao što su otkazivanje fizičkog servera, prekid mrežne komunikacije ili potreba za migracijom virtualne mašine. U takvim slučajevima skaliranje i preraspodjela resursa moraju biti koordinirani kako bi se očuvao kontinuitet pružanja usluge.

Nisu sve aplikacije podjednako pogodne za dinamičko skaliranje. Aplikacija mora biti projektovana tako da se njen rad može rasporediti na dodatne izvršne jedinice ili da može efikasno koristiti povećane kapacitete postojeće jedinice. Kod aplikacija čije se opterećenje može dinamički podijeliti, dodatne instance mogu preuzeti dio zahtjeva i omogućiti povećanje ukupnog kapaciteta. Kod aplikacija sa čvrsto povezanim komponentama ili statički podijeljenim poslovima mogućnosti skaliranja mogu biti ograničene [19].

Pored tehničkih karakteristika aplikacije, važno je uzeti u obzir troškove skaliranja. Pokretanje, zaustavljanje i migracija virtualnih mašina stvaraju određeno opterećenje i mogu privremeno koristiti procesorske, memorijske, skladišne i mrežne resurse. Prečesto skaliranje može povećati složenost upravljanja i proizvesti troškove koji poništavaju dio koristi ostvarene prilagođavanjem kapaciteta. Zbog toga politika skaliranja mora razlikovati kratkotrajne promjene od trajnijih promjena opterećenja.

Značaj dinamičkog skaliranja ogleda se u njegovom doprinosu performansama, dostupnosti i ekonomičnosti cloud sistema. Pravilno skaliranje omogućava aplikacijama da održe prihvatljivo vrijeme odziva i u periodima povećanog broja zahtjeva. Istovremeno, uklanjanjem nepotrebnih resursa smanjuju se troškovi i povećava iskorištenost infrastrukture. Na nivou pružaoca usluge, oslobođeni kapaciteti mogu biti dodijeljeni drugim aplikacijama, čime se unapređuje ukupna efikasnost podatkovnog centra [7], [8].

Dinamičko skaliranje stoga predstavlja jedan od najvažnijih mehanizama optimizacije resursa u cloud okruženju. Njegova primjena omogućava usklađivanje količine dodijeljenih resursa sa stvarnim opterećenjem, smanjuje potrebu za trajnim rezervisanjem dodatnih kapaciteta i doprinosi ostvarivanju ravnoteže između performansi i troškova. Konkretni način skaliranja zavisi od arhitekture aplikacije i vrste raspoloživih resursa, pri čemu se osnovni pristupi mogu podijeliti na horizontalno i vertikalno skaliranje.

5.2. Horizontalno i vertikalno skaliranje

Upravljanje promjenjivim opterećenjem u cloud okruženju najčešće se zasniva na dvije osnovne strategije: horizontalnom i vertikalnom skaliranju. Obje strategije imaju isti opći cilj, a to je prilagođavanje količine raspoloživih resursa stvarnim potrebama aplikacije. Razlika između njih ogleda se u načinu na koji se dodatni kapacitet obezbjeđuje. Kod vertikalnog skaliranja povećava se kapacitet postojećih izvršnih jedinica, dok se kod horizontalnog skaliranja povećava njihov broj [7], [19].

Izbor između horizontalnog i vertikalnog skaliranja zavisi od arhitekture aplikacije, vrste radnog opterećenja, mogućnosti podjele zadataka, dostupnosti fizičke infrastrukture i vremena potrebnog za izvršavanje promjene. Pored tehničkih karakteristika, potrebno je uzeti u obzir i troškove skaliranja, potrošnju energije, mrežni saobraćaj i zahtjeve u pogledu kvaliteta usluge.

Vertikalno skaliranje podrazumijeva da broj virtualnih mašina koje izvršavaju aplikaciju ostaje nepromijenjen, dok se povećava ili smanjuje količina resursa dodijeljenih svakoj virtualnoj mašini. Dodatni kapacitet može obuhvatiti procesorsko vrijeme, broj procesorskih

jezgri, memoriju, skladišni prostor i ulazno-izlazne kapacitete. Ovaj pristup se često označava kao scale up kada se resursi povećavaju, odnosno scale down kada se smanjuju.

Povećanje kapaciteta virtualne mašine može se realizovati na dva osnovna načina. Prvi način podrazumijeva zadržavanje virtualne mašine na istom fizičkom serveru, ali povećavanje njenog udjela u procesorskom vremenu ili drugim raspoloživim resursima. Ovakav pristup moguć je samo ako fizički server raspolaže slobodnim kapacitetima koji se mogu dodijeliti virtualnoj mašini.

Drugi način vertikalnog skaliranja podrazumijeva migraciju virtualne mašine na snažniji fizički server. Migracija može biti potrebna kada postojeći server nema dovoljno slobodnih procesorskih, memorijskih ili drugih kapaciteta. U tom slučaju virtualna mašina se zaustavlja, kreira se snimak njenog stanja, podaci se prenose na odabrani server, a zatim se virtualna mašina ponovo pokreće na novoj lokaciji [1].

Migracija virtualne mašine proizvodi dodatno opterećenje sistema. Prenos njenog stanja može zahtijevati značajan mrežni kapacitet i vrijeme, posebno ako virtualna mašina koristi veću količinu memorije ili skladišnog prostora. Tokom migracije može doći i do privremenog prekida ili smanjenja performansi aplikacije. Zbog toga se odluka o migraciji mora donositi uz procjenu očekivane koristi i troškova samog postupka.

Prednost vertikalnog skaliranja ogleda se u tome što se aplikacija može izvršavati na manjem broju virtualnih mašina. Time se smanjuje potreba za komunikacijom između više instanci, pojednostavljuje upravljanje aplikacijom i izbjegava složena raspodjela podataka i zadataka. Ovaj pristup može biti posebno pogodan za aplikacije kod kojih je radno opterećenje teško podijeliti na više nezavisnih dijelova.

Vertikalno skaliranje ipak ima ograničenja. Svaki fizički server raspolaže konačnom količinom procesorske snage, memorije i skladišnog prostora, zbog čega se kapacitet pojedinačne virtualne mašine ne može povećavati neograničeno. Kada se dostigne maksimalni kapacitet servera, daljnje vertikalno skaliranje zahtijeva migraciju na snažniji sistem ili prelazak na horizontalni pristup.

Dodatni nedostatak može biti veća zavisnost od jedne virtualne mašine ili fizičkog servera. Ako aplikacija koristi samo jednu snažnu instancu, kvar te instance može imati veći uticaj na dostupnost usluge. Zbog toga vertikalno skaliranje samo po sebi ne rješava pitanja otpornosti na kvarove i visoke dostupnosti, nego se mora kombinovati sa rezervnim instancama, replikacijom ili drugim mehanizmima zaštite.

Horizontalno skaliranje podrazumijeva povećavanje broja virtualnih mašina ili instanci aplikacije kada opterećenje raste, odnosno njihovo smanjivanje kada opterećenje opada. Dodavanje novih instanci označava se kao scale out, dok se njihovo uklanjanje označava kao scale in. U knjizi se horizontalno skaliranje navodi kao najčešće korištena strategija u cloud okruženju [1], [19].

Kod horizontalnog skaliranja korisnički zahtjevi ili zadaci raspoređuju se između više aktivnih instanci. Time se omogućava paralelna obrada većeg broja zahtjeva i povećava ukupni kapacitet aplikacije. Kada se broj zahtjeva smanji, dio instanci može se zaustaviti kako bi se smanjili troškovi i oslobodili resursi za druge aplikacije.

Važnu ulogu u horizontalnom skaliranju ima balansiranje opterećenja. Balancer opterećenja prima dolazne zahtjeve i prosljeđuje ih odgovarajućim instancama aplikacije. Njegov cilj jeste spriječiti situaciju u kojoj su pojedine instance preopterećene, dok su druge nedovoljno iskorištene. Ravnomjerna raspodjela zahtjeva omogućava bolju iskorištenost resursa i stabilnije vrijeme odziva [22], [23].

U transakcijski orijentisanim sistemima raspodjelu zahtjeva često izvršava prednji, odnosno front-end server, koji zahtjeve prosljeđuje većem broju pozadinskih servera. Kod veoma velikih aplikacija jedan balancer opterećenja može postati usko grlo, pa je moguće koristiti više međusobno povezanih balansera. Njihova koordinacija tada postaje dodatni zadatak sistema za upravljanje resursima.

Horizontalno skaliranje može povećati mrežni saobraćaj aplikacije. Veći broj virtualnih mašina često zahtijeva razmjenu podataka, sinhronizaciju stanja i pristup zajedničkom skladištu. Ako instance aplikacije intenzivno međusobno komuniciraju, troškovi komunikacije mogu smanjiti korist ostvarenu dodavanjem novih resursa. Zbog toga raspoređivanje instanci mora uzeti u obzir njihovu fizičku lokaciju i karakteristike mrežne infrastrukture.

Primjena horizontalnog skaliranja zavisi od toga da li se radno opterećenje aplikacije može dinamički podijeliti. Kod modularno djeljivih aplikacija podjela zadataka određuje se unaprijed i ne može se jednostavno mijenjati tokom izvršavanja. U takvim slučajevima mogućnosti horizontalnog skaliranja mogu biti ograničene, pa vertikalno skaliranje predstavlja pogodniji pristup.

Kod proizvoljno djeljivih aplikacija opterećenje se može dinamički raspoređivati na dodatne virtualne mašine. Kada se intenzitet zahtjeva poveća, nove instance preuzimaju dio radnog

opterećenja. Kada se zahtjevi smanje, nepotrebne instance se uklanjaju. Većina cloud aplikacija projektuje se tako da podržava ovaj oblik raspodjele, zbog čega je horizontalno skaliranje široko zastupljeno.

Tipični primjeri aplikacija pogodnih za horizontalno skaliranje jesu web aplikacije i sistemi za obradu velikog broja nezavisnih korisničkih zahtjeva. Ako svaki zahtjev može biti obrađen relativno nezavisno od ostalih, njegovo usmjeravanje na jednu od više dostupnih instanci ne zahtijeva složenu koordinaciju. Time se kapacitet sistema može povećavati postepenim dodavanjem novih instanci.

Aplikacije koje čuvaju značajnu količinu lokalnog stanja složenije su za horizontalno skaliranje. Ako su podaci korisničke sesije vezani isključivo za jednu instancu, zahtjevi istog korisnika moraju se usmjeravati prema istoj virtualnoj mašini ili se stanje mora premjestiti u zajednički sistem za pohranu. Zbog toga se skalabilne cloud aplikacije često projektuju tako da izvršne instance budu što manje zavisne od lokalnog stanja.

Prilikom izbora načina skaliranja potrebno je uzeti u obzir i karakter radnog opterećenja. Aplikacije koje intenzivno koriste komunikacijske resurse mogu imati koristi od izvršavanja na snažnijem serveru, jer se time smanjuje potreba za prenosom podataka između većeg broja fizičkih sistema. Iako cijena procesorskog vremena na snažnijem serveru može biti veća, kraće vrijeme izvršavanja i manji mrežni saobraćaj mogu smanjiti ukupne troškove korisnika i pružaoca usluge.

Kod procesorski intenzivnih aplikacija povećavanje ili smanjivanje broja virtualnih mašina može biti efikasna strategija, pod uslovom da se radno opterećenje može paralelno obrađivati. Ako je intenzitet opterećenja relativno stabilan, troškovi pokretanja ili zaustavljanja pojedinačnih instanci neće imati veliki uticaj na ukupno vrijeme izvršavanja.

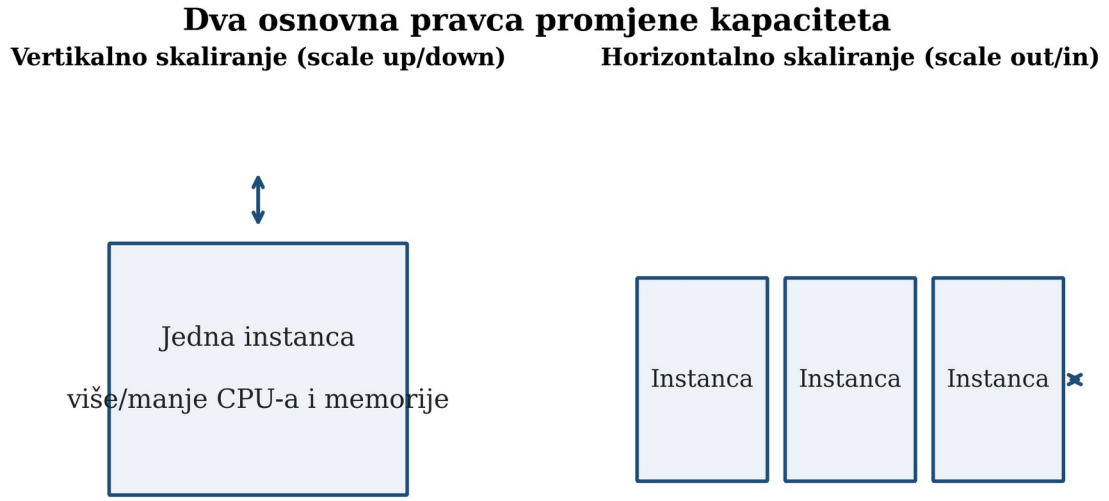
Vertikalno i horizontalno skaliranje ne moraju se posmatrati kao međusobno isključivi pristupi. U praksi se mogu kombinovati. Sistem može najprije povećavati resurse postojećih virtualnih mašina, a zatim, kada se približe granicama dostupnog kapaciteta, dodavati nove instance. Slično tome, nakon smanjenja opterećenja moguće je prvo ukloniti dio instanci, a zatim smanjiti resurse koji su dodijeljeni preostalim virtualnim mašinama [7], [19].

Kombinovani pristup može omogućiti bolje iskorištenje infrastrukture, ali istovremeno povećava složenost upravljanja. Sistem mora odlučiti da li je u određenom trenutku povoljnije povećati kapacitet postojeće instance ili pokrenuti novu instancu. Odluka zavisi od

dostupnosti resursa, vremena pokretanja virtualne mašine, mrežnih zahtjeva aplikacije, troškova i očekivanog trajanja povećanog opterećenja.

Može se zaključiti da je vertikalno skaliranje jednostavnije za aplikacije koje se teško dijele i koje imaju velike zahtjeve za međusobnom komunikacijom komponenti. Horizontalno skaliranje pogodnije je za aplikacije čije se opterećenje može rasporediti na veći broj nezavisnih instanci. Zbog mogućnosti postepenog dodavanja i uklanjanja virtualnih mašina, horizontalni pristup predstavlja osnovu elastičnosti velikog broja savremenih cloud aplikacija.

Efikasna primjena oba pristupa zahtijeva kontinuirano praćenje stanja sistema i pravovremeno donošenje odluka. Ručno upravljanje velikim brojem virtualnih mašina nije praktično, posebno u sistemima sa brzim i nepredvidivim promjenama opterećenja. Zbog toga se horizontalno i vertikalno skaliranje najčešće povezuju sa automatizovanim mehanizmima nadzora, odlučivanja i izvršavanja upravljačkih akcija, koji će biti detaljnije razmotreni u narednom potpoglavlju.



Slika 5.1. Horizontalno i vertikalno skaliranje

Izvor: izrada autora prema [1], [7] i [19].

5.3. Automatizovano upravljanje resursima

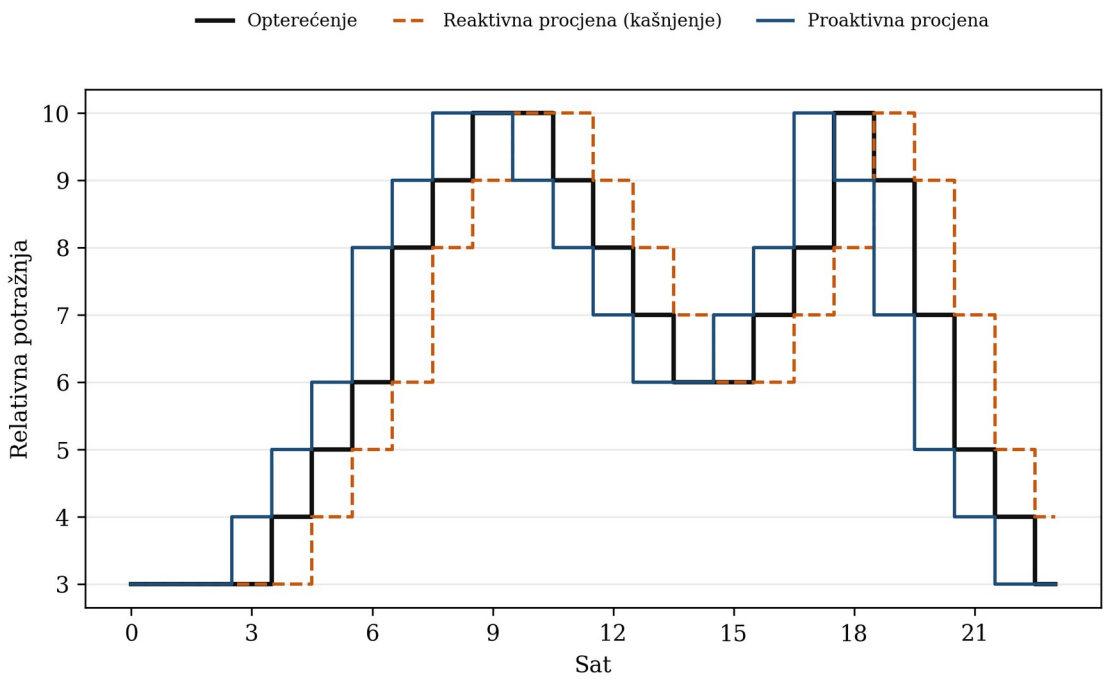
Automatizovano upravljanje resursima predstavlja prelazak sa povremenih, ručnih administrativnih intervencija na kontinuirani proces u kojem sistem samostalno posmatra stanje infrastrukture, procjenjuje odstupanje od postavljenih ciljeva i izvršava korektivne akcije. Takav proces je potreban zbog obima cloud infrastrukture i brzine promjena

opterećenja: odluke koje su prihvatljive za nekoliko servera postaju neodržive kada sistem obuhvata hiljade čvorova i veliki broj heterogenih aplikacija [1], [13].

Osnovni model automatizovanog upravljanja može se opisati zatvorenom upravljačkom petljom. Prva faza je nadzor, u kojem se prikupljaju podaci o iskorištenosti procesora i memorije, mrežnom saobraćaju, dužini redova, vremenu odziva, broju grešaka i troškovima. Nakon toga slijedi analiza: izmjerene vrijednosti porede se sa pragovima, ciljevima kvaliteta usluge ili predviđenim budućim stanjem. Planer zatim bira akciju, a izvršni sloj mijenja broj instanci, njihove resurse ili raspored zadataka. Rezultat akcije ponovo se mjeri, čime se petlja zatvara [7], [8].

Kvalitet automatizacije zavisi od kvaliteta telemetrije. Jedna metrika rijetko je dovoljna za pouzdanu odluku. Visoka iskorištenost procesora može biti normalna kod računarski intenzivnog batch posla, ali problematična kod web usluge sa strogim ciljem vremena odziva. S druge strane, niska prosječna iskorištenost ne isključuje kratke vršne intervale koji uzrokuju kašnjenja. Zbog toga se u praksi kombinuju infrastrukturne metrike, aplikacijske metrike i pokazatelji kvaliteta usluge.

Pravila zasnovana na pragovima predstavljaju najjednostavniji oblik automatizacije. Primjer je dodavanje instance kada prosječna iskorištenost procesora pređe 70% tokom pet minuta i njeno uklanjanje kada iskorištenost ostane ispod 30% tokom dužeg intervala. Različiti pragovi za povećavanje i smanjivanje kapaciteta uvode histerezu i sprečavaju oscilovanje sistema. Dodatno se koriste periodi stabilizacije i ograničenja brzine promjene kako kratkotrajni skok ne bi pokrenuo niz skupih akcija [7], [22].



Slika 5.2. Reaktivno i proaktivno skaliranje

Izvor: izrada autora; konceptualni prikaz prema [7], [8] i [19].

U složenijim sistemima odluke se donose pomoću kontrolne teorije, teorije redova čekanja, vremenskih serija ili metoda mašinskog učenja. Kontrolni modeli nastoje održavati ciljnu vrijednost, na primjer vrijeme odziva, mijenjanjem dodijeljenog kapaciteta. Prediktivni modeli procjenjuju buduće opterećenje i resurse pripremaju prije nastanka vrha. Njihova prednost je kraće kašnjenje reakcije, ali pogrešna prognoza može dovesti do prekomjerne ili nedovoljne alokacije [7], [8], [19].

Automatizacija mora obuhvatiti i upravljanje greškama. Nova instanca nije korisna samo zato što je pokrenuta; potrebno je provjeriti njeno zdravstveno stanje, učitati konfiguraciju, povezati je sa servisima i tek tada uključiti u obradu saobraćaja. Pri uklanjanju instance treba završiti aktivne zahtjeve i sačuvati stanje koje ne smije biti izgubljeno. Zbog toga je vrijeme aktiviranja resursa sastavni dio politike, a ne sporedna tehnička pojedinost.

Kontejnerske platforme automatizaciju provode na više nivoa. Horizontalno skaliranje mijenja broj izvršnih jedinica, vertikalno mijenja njihove zahtjeve i ograničenja, dok skaliranje klastera dodaje ili uklanja radne čvorove. Kubernetes Horizontal Pod Autoscaler, na primjer, periodično poredi izmjerenu metriku sa željenom vrijednošću i izračunava potreban broj replika; scheduling framework zatim bira čvor na kojem će nova replika biti izvršena [20], [21].

Višenivojsko upravljanje može proizvesti konflikt. Aplikacijski autoscaler može povećati broj replika upravo u trenutku kada infrastrukturni sloj pokušava ugasiti nedovoljno iskorišten server. Slično tome, vertikalno i horizontalno skaliranje mogu istovremeno reagovati na istu metriku. Koordinacija zato zahtijeva jasno definisanu nadležnost, različite vremenske horizonte i zajedničke ciljeve, kao što su granice troška, vrijeme odziva i minimalna raspoloživost [14], [15].

Automatizacija nije sama sebi cilj. Dobra politika mora biti objašnjiva, mjerljiva i sigurna. Potrebno je evidentirati razlog svake akcije, ograničiti maksimalni i minimalni kapacitet, omogućiti ručnu intervenciju i redovno provjeravati da li su ciljevi ostvareni. Time se upravljanje resursima pretvara u kontrolisan proces kontinuiranog poboljšavanja, a ne u skup nepovezanih skripti.

Tabela 5.1. Poređenje pristupa autoscalingu

Pristup	Osnova odluke	Prednost	Glavni rizik
Reaktivni	Trenutne i nedavne metrike	Jednostavan i robustan	Kašnjenje reakcije
Proaktivni	Prognoza budućeg opterećenja	Kapacitet prije vrha	Greška predviđanja
Hibridni	Prognoza i reaktivna korekcija	Pokriva obrazac i odstupanje	Konflikt kontrolera
Rasporedni	Poznati kalendar događaja	Predvidiv i transparentan	Ne reaguje na iznenađenja

Izvor: izrada autora prema [7], [8], [19] i [23].

5.4. Maksimiziranje protoka sistema

Protok sistema označava količinu korisnog rada završenu u posmatranom vremenskom intervalu. Može se izražavati brojem obrađenih zahtjeva, završenih poslova, transakcija ili jedinica podataka. Pojednostavljeno, protok se može zapisati kao $X = N / \Delta t$, gdje je N broj završenih zadataka, a Δt trajanje posmatranog intervala. Maksimiziranje protoka ne znači samo dodavanje procesorske snage; zahtijeva uklanjanje uskih grla duž čitavog lanca izvršavanja.

Scheduler utiče na protok izborom zadataka i mjesta izvršavanja. Ako se zadaci rasporede na čvorove koji ne posjeduju potrebne podatke, dio vremena i mrežnog kapaciteta troši se na prenos. Data-aware scheduling nastoji izvršavanje približiti podacima i tako smanjiti komunikacijski trošak. Delay scheduling pokazuje da kratko odlaganje zadatka može povećati lokalnost podataka bez značajnog narušavanja pravičnosti [11], [12].

Povećanje paralelizma poboljšava protok samo dok aplikacija može efikasno koristiti dodatne izvršne jedinice. Nakon određene tačke koordinacija, sinhronizacija, zaključavanje i mrežna komunikacija postaju dominantni. Zbog toga optimalan broj instanci zavisi od odnosa računanja i komunikacije, veličine zadataka i dostupnosti zajedničkih servisa kao što su baze podataka.

Grupisanje kompatibilnih radnih opterećenja može povećati iskorištenost servera. Googleov Borg ostvaruje visoku iskorištenost kombinovanjem kontrole prijema, pakovanja zadataka, dijeljenja mašina i kontrolisanog prekomjernog rezervisanja, uz izolaciju performansi između procesa [13]. Ovakav pristup pokazuje da je protok rezultat koordinacije više mehanizama, a ne djelovanja jednog algoritma.

Mjerenje protoka mora se tumačiti zajedno sa kvalitetom usluge. Sistem može povećati ukupan broj završenih batch zadataka, a istovremeno pogoršati vrijeme odziva interaktivnih aplikacija. Zbog toga se protok obično optimizira uz ograničenja: maksimalno vrijeme odziva, dopuštenu stopu grešaka, prioritetne klase i minimalni udio resursa za svakog korisnika.

Praktične mjere za povećanje protoka uključuju uklanjanje najizraženijeg uskog grla, ograničavanje zadataka koji stvaraju zagušenje, lokalnost podataka, balansiranje opterećenja i skaliranje sloja koji je stvarno zasićen. Selektivno skaliranje je obično efikasnije od istovremenog povećavanja svih komponenti aplikacije.

5.5. Minimiziranje vremena obrta

Vrijeme obrta predstavlja interval od podnošenja zadatka do njegovog završetka: T_obrta = t_završetka - t_podnošenja. Sastoji se od vremena čekanja u redu, vremena potrebnog za pripremu okruženja, stvarnog vremena izvršavanja i eventualnih pauza ili ponovnih pokušaja. Zbog toga se ne može pouzdano smanjiti optimiziranjem samo jedne faze.

Kratki zadaci mogu dugo čekati iza velikih poslova iako zahtijevaju malo procesorskog vremena. Politike koje uzimaju u obzir procijenjeno trajanje mogu smanjiti prosječno vrijeme

obrta, ali procjena nije uvijek tačna. Prioritetno planiranje poboljšava odziv kritičnih poslova, dok mehanizam starenja sprečava neograničeno odlaganje zadataka nižeg prioriteta.

Preemptivno planiranje omogućava oslobađanje resursa za hitniji zadatak, ali uvodi trošak zaustavljanja, čuvanja stanja i kasnijeg nastavka. Preempcija je opravdana kada je očekivana korist veća od upravljačkog troška i kada aplikacija može sigurno nastaviti izvršavanje. Za kratke zadatke sa velikim troškom ponovnog pokretanja nepreemptivni pristup može biti povoljniji.

Vrijeme aktiviranja resursa posebno je značajno kod autoscalinga. Ako je za pokretanje virtualne mašine potrebno nekoliko minuta, reaktivna politika može zakasniti u odnosu na nagli porast zahtjeva. Topli skup unaprijed pripremljenih instanci, kontejneri sa kraćim vremenom pokretanja i prediktivno skaliranje mogu smanjiti ovaj dio vremena obrta [7], [19].

Kod distribuiranih workflow aplikacija vrijeme obrta zavisi i od zavisnosti između zadataka. Dodavanje resursa nezavisnom zadatku koji nije na kritičnom putu neće skratiti završetak cijelog workflowa. Efikasna alokacija zato prepoznaje kritični put, rok i odnos između cijene dodatnog resursa i očekivanog skraćenja izvršavanja [9], [18].

Optimizacija vremena obrta mora razlikovati prosjek od repa distribucije. Prosječna vrijednost može izgledati prihvatljivo iako mali dio zahtjeva doživljava veoma velika kašnjenja. Za korisničke servise zato se prate percentili, na primjer 95. ili 99. percentil vremena odziva, i upravo se prema njima često definišu ciljevi kvaliteta usluge.

5.6. Optimizacija troškova korištenja cloud resursa

Trošak korištenja cloud resursa nastaje iz više komponenti: procesorskog vremena, memorije, skladišta, operacija nad podacima, mrežnog prenosa, licenci i upravljačkih servisa. Pojednostavljen model ukupnog troška može se izraziti kao $C = \Sigma(p_i \times q_i \times t_i)$, gdje p_i predstavlja jediničnu cijenu, q_i količinu resursa, a t_i trajanje korištenja. Model pokazuje da trošak zavisi i od veličine i od vremena alokacije.

Najčešći izvor nepotrebnog troška je trajno rezervisanje kapaciteta za rijetke vršne vrijednosti. Takav sistem veći dio vremena radi sa niskom iskorištenošću, iako se puni kapacitet plaća. Dinamičko skaliranje smanjuje jaz između raspoloživog i stvarno potrebnog kapaciteta, ali samo ako su minimalne i maksimalne granice, pragovi i periodi stabilizacije pravilno postavljeni [7], [22].

Smanjivanje resursa ima granicu određenu kvalitetom usluge. Agresivno uklanjanje instanci može prouzrokovati kašnjenja, greške i povredu SLA-a, čiji poslovni trošak može biti veći od uštede na infrastrukturi. Zbog toga cost-aware politika ne minimizira račun izolovano, nego traži najmanji trošak uz prihvatljivu dostupnost, vrijeme odziva i rizik.

Pravilno dimenzioniranje podrazumijeva izbor tipa i veličine instance prema profilu opterećenja. Memorijski intenzivna aplikacija ne dobija mnogo dodavanjem procesorskih jezgri, dok procesorski posao može biti skuplji na instanci sa velikom količinom neiskorištene memorije. Kontinuirano poređenje rezervisanog i korištenog kapaciteta omogućava vertikalno prilagođavanje i uklanjanje trajno prevelikih konfiguracija.

Model kupovine također utiče na odluke. Dugotrajno stabilno opterećenje može opravdati rezervisani kapacitet ili ugovorenu potrošnju, dok se promjenjivi dio opterećenja pokriva elastičnim resursima. Zadaci tolerantni na prekid mogu koristiti jeftiniji, ali nepouzdaniji višak kapaciteta. Kombinovanje modela smanjuje trošak samo ako scheduler poznaje zahtjeve zadataka i posljedice prekida.

Troškovi prenosa podataka često se zanemaruju pri optimizaciji procesorskih resursa. Premještanje velike količine podataka između zona ili regija može poništiti uštedu postignutu jeftinijom instancom. Data-aware scheduling, keširanje i zajednička lokacija međuzavisnih komponenti smanjuju i mrežni trošak i vrijeme obrade [12].

Upravljanje troškovima zahtijeva jasnu raspodjelu odgovornosti. Tehničke metrike treba povezati sa poslovnim vlasnikom, uslugom i okruženjem. Označavanje resursa, budžetski pragovi i redovni izvještaji omogućavaju uočavanje napuštenih diskova, neaktivnih instanci i razvojnih okruženja koja ostaju uključena izvan potrebnog vremena. AWS Well-Architected okvir ovaj proces posmatra kao kontinuiranu disciplinu, a ne jednokratno smanjenje računa [22].

Automatizovane akcije uštede moraju biti provjerljive. Preporučljivo je najprije posmatrati učinak predložene promjene, zatim ograničiti njen obim i definisati mogućnost povratka. Time se sprečava da kratkoročna finansijska optimizacija naruši pouzdanost sistema ili stvori skriveni operativni trošak.

5.7. Smanjenje potrošnje energije u cloud okruženju

Potrošnja energije podatkovnog centra obuhvata računarsku opremu, mrežne i skladišne uređaje, hlađenje i pomoćne sisteme. Iako je rast digitalnih usluga velik, poboljšanja

energetske efikasnosti ograničila su rast ukupne globalne potrošnje podatkovnih centara u posmatranom periodu 2010-2018, što pokazuje značaj tehnološke i upravljačke efikasnosti [17].

Server u stanju mirovanja ne troši nultu energiju. Zbog toga veliki broj slabo iskorištenih servera može imati veću ukupnu potrošnju od manjeg broja dobro opterećenih servera. Konsolidacija premješta virtualne mašine ili kontejnere na manji broj fizičkih čvorova, nakon čega se oslobođeni čvorovi mogu prebaciti u stanje manje potrošnje ili ugasiti [6].

Konsolidacija ne smije biti preagresivna. Prevelika gustina zadataka povećava interferenciju, temperaturu i rizik da kratki porast opterećenja naruši kvalitet usluge. Energetski svjesna alokacija zato uzima u obzir iskorištenost više resursa, istoriju opterećenja i trošak migracije, a ne samo trenutnu vrijednost procesora [6], [16].

Dinamičko upravljanje naponom i frekvencijom procesora može smanjiti potrošnju kada puna brzina nije potrebna. Međutim, smanjena frekvencija produžava izvršavanje zadatka i može povećati energiju drugih komponenti. Odluka zato treba posmatrati ukupnu energiju po završenoj jedinici rada, a ne samo trenutnu snagu.

Lokacija izvršavanja također može biti dio energetske politike. Podatkovni centri razlikuju se prema efikasnosti hlađenja, trenutnom opterećenju i ugljičnom intenzitetu električne energije. Poslovi tolerantni na odlaganje mogu se vremenski ili geografski pomjeriti, ali se moraju uzeti u obzir mrežni trošak, propisi o lokaciji podataka i rok završetka.

Jedna od često korištenih infrastrukturnih mjera je PUE (Power Usage Effectiveness), odnos ukupne energije objekta i energije IT opreme. PUE je koristan za procjenu pomoćne potrošnje, ali ne pokazuje koliko korisnog rada aplikacija obavlja. Zato ga treba dopuniti aplikacijskim pokazateljima kao što su energija po transakciji ili po obrađenoj jedinici podataka.

Energetska i troškovna optimizacija često su usklađene jer smanjenje broja nepotrebno aktivnih resursa umanjuje i račun i potrošnju. Ipak, cijena električne energije, ugovorni modeli i ugljični intenzitet nisu ista mjera. Politika mora jasno odrediti optimizira li finansijski trošak, potrošnju energije ili emisije.

Najbolji rezultati postižu se kombinovanjem energetski efikasnog hardvera, konsolidacije, autoscalinga, pažljivog schedulinga i mjerenja na nivou aplikacije. Energetska efikasnost stoga nije poseban dodatak upravljanju resursima, nego jedan od ciljeva iste višekriterijske optimizacije.

6. Analiza uticaja upravljanja resursima i njihove alokacije u oblaku

U prethodnim poglavljima razmotreni su pojmovi, politike i mehanizmi upravljanja resursima. Ovo poglavlje povezuje te nalaze sa postavljenim hipotezama. Analiza je kvalitativna i komparativna, uz jedan transparentan ilustrativni proračun. Zbog toga se zaključci odnose na uslove pod kojima određeni pristup može dati korist, a ne na tvrdnju da je jedan algoritam univerzalno najbolji.

6.1. Uticaj kvalitetnog upravljanja resursima na troškove cloud usluga

Trošak cloud usluge zavisi od razlike između kapaciteta koji je aktiviran i kapaciteta koji obavlja koristan rad. Statička alokacija projektovana prema vršnom opterećenju pruža rezervu i jednostavnost, ali u periodima manjeg opterećenja stvara neiskorišten kapacitet. Elastična alokacija smanjuje ovu razliku prilagođavanjem broja ili veličine resursa stvarnoj potražnji [7], [8].

Ušteda ipak nije automatska posljedica uključivanja autoscalinga. Ako su pragovi preniski, sistem može često dodavati resurse zbog kratkih skokova. Ako su previsoki, kasna reakcija povećava vrijeme odziva i rizik povrede SLA-a. Prečesto skaliranje uvodi trošak pokretanja, migracije i upravljanja. Efekat zato zavisi od stabilnosti politike i odnosa između vremena reakcije i dinamike opterećenja.

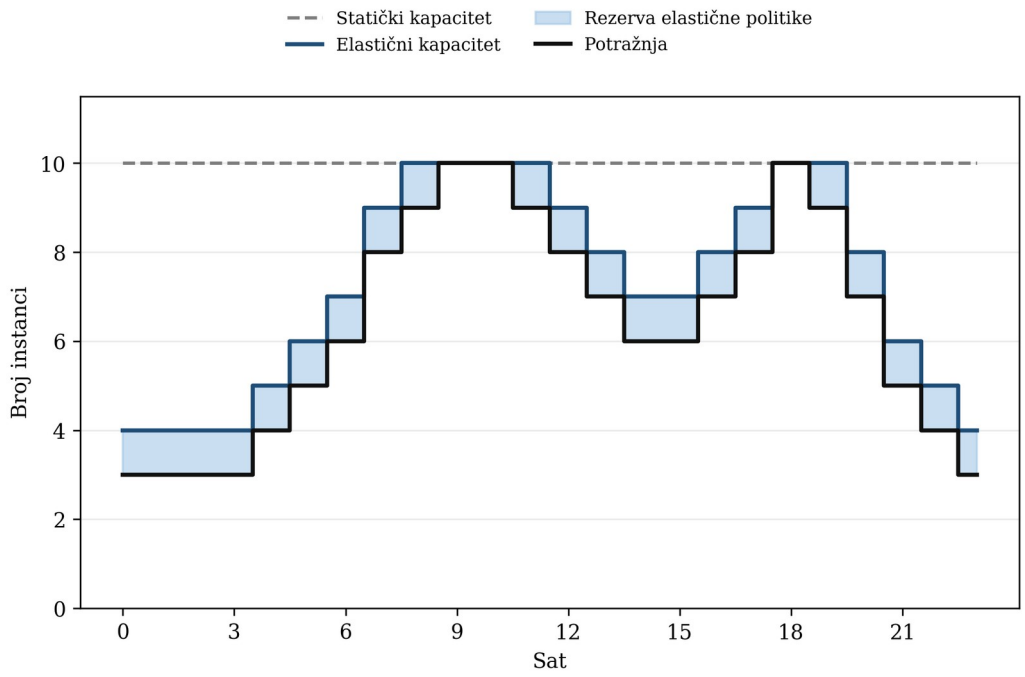
Ilustrativni proračun u ovom radu posmatra 24 jednosatna intervala. Statička politika održava deset instanci, koliko je potrebno u vršnom intervalu. Elastična politika u svakom intervalu aktivira procijenjenu potražnju uvećanu za jednu rezervnu instancu, uz gornju granicu od deset. Pretpostavlja se ista jedinična cijena po instanci i satu, bez rezervacijskih popusta, mrežnog troška i vremena pokretanja. Pretpostavke su namjerno jednostavne kako bi posljedica odluke bila provjerljiva.

Statička politika u tom scenariju koristi 240 instanca-sati. Elastična politika koristi 174 instanca-sata, što je 27,5% manje aktiviranog kapaciteta. Koristan zahtijevani rad iznosi 153 instanca-sata u oba slučaja. Odnos korisnog i aktiviranog kapaciteta raste sa 63,8% kod statičke na 87,9% kod elastične politike. Rezultat ne predstavlja tržišnu cijenu niti produkcijsko mjerenje, nego demonstrira mehanizam nastanka uštede.

U stvarnom sistemu elastična politika mora zadržati dodatnu rezervu zbog grešaka prognoze i vremena pokretanja. Ušteda bi mogla biti manja zbog minimalnog broja instanci, naplate

započetog intervala, licenci ili prijenosa podataka. Mogla bi biti i veća ako se kombinuju prava veličina instanci, gašenje razvojnih okruženja i povoljniji modeli kupovine. Zbog toga se ilustrativni procenat ne generalizira, dok je pravac efekta u skladu sa literaturom o elastičnosti i troškovno svjesnom skaliranju [7], [9], [22].

Najvažniji zaključak analize troška jeste da politika mora optimizirati ukupni trošak usluge, a ne samo broj instanci. U ukupnu odluku ulaze rizik prekida, rad administratora, mrežni prenos, pohrana, licence i poslovna vrijednost raspoloživosti. Minimalna infrastrukturna potrošnja nije optimalna ako dovodi do gubitka zahtjeva ili čestih incidenata.



Slika 6.1. Ilustrativno poređenje potražnje i alociranog kapaciteta

Izvor: izrada autora; ilustrativni scenarij.

Tabela 6.1. Ilustrativni proračun statičke i elastične alokacije

Pokazatelj	Statička politika	Elastična politika
Aktivirani kapacitet	240 instanca-sati	174 instanca-sata
Zahtijevani koristan rad	153 instanca-sata	153 instanca-sata
Odnos korisnog i aktiviranog	63,8%	87,9%
Promjena aktiviranog kapaciteta	Referentna vrijednost	-27,5%

Izvor: izračun autora na osnovu jasno navedenih ilustrativnih pretpostavki.

6.2. Uticaj kvalitetne alokacije resursa na efikasnost korištenja IT resursa

Efikasnost korištenja IT resursa može se posmatrati kroz iskorištenost, protok, vrijeme obrta, lokalnost podataka, udio uspješno završenih zadataka i poštivanje ciljeva kvaliteta usluge. Visoka iskorištenost sama po sebi nije dovoljna: sistem koji stalno radi blizu fizičke granice može imati lošije vrijeme odziva i veću stopu grešaka. Efikasnost zato znači ostvariti željeni nivo usluge uz najmanji razumno potreban kapacitet.

Scheduling povećava efikasnost kada raspoređuje zadatke prema njihovim stvarnim zahtjevima. Pakovanje zadataka različitog profila može popuniti resurse koje bi homogeno opterećenje ostavilo neiskorištenim. Borg koristi dijeljenje mašina, kontrolu prijema i prekomjerno rezervisanje kako bi povećao iskorištenost, uz mehanizme izolacije koji ograničavaju interferenciju [13], [16].

Pravična alokacija je posebno važna u višekorisničkom okruženju. Jednaka raspodjela procesora nije nužno pravična kada zadaci koriste različite kombinacije procesora i memorije. Dominant Resource Fairness posmatra najveći relativni udio svakog korisnika u odnosu na različite resurse, čime proširuje ideju pravičnosti na višedimenzionalnu alokaciju [10].

Lokalnost podataka smanjuje mrežnu komunikaciju i oslobađa kapacitet za druge zadate. Quincy uključuje lokaciju podataka u optimizaciju rasporeda, dok delay scheduling kratkim čekanjem povećava vjerovatnoću lokalnog izvršavanja [11], [12]. Ovi pristupi pokazuju da efikasnost nije određena samo količinom dodijeljenih resursa, nego i mjestom i vremenom njihove dodjele.

Automatizovano skaliranje poboljšava efikasnost kada smanjuje trajanje perioda prekomjerne i nedovoljne alokacije. Reaktivni pristup je robustan i jednostavniji, ali reaguje nakon promjene. Proaktivni pristup skraćuje kašnjenje ako je prognoza tačna. Hibridni pristup koristi predviđanje za osnovni plan, a reaktivna pravila za neočekivano odstupanje [7], [8], [19].

Efikasnost mora biti mjerena na odgovarajućem vremenskom horizontu. Kratak interval otkriva vrhove, ali može uzrokovati nestabilne odluke; dug interval izglađuje šum, ali prikriva probleme. Zato se često kombinuju brze metrike za zaštitu kvaliteta usluge i sporiji trendovi za promjenu osnovnog kapaciteta.

6.3. Komparativna analiza različitih pristupa upravljanju resursima

Komparativna analiza obuhvata statičku alokaciju, reaktivno skaliranje, proaktivno skaliranje i hibridno upravljanje. Kriteriji poređenja su složenost, vrijeme reakcije, otpornost na

neočekivane promjene, rizik prekomjerne alokacije, mogućnost optimizacije troška i zahtjevi za podacima.

Statička alokacija ima najmanju upravljačku složenost i predvidiv kapacitet. Pogodna je za stabilno opterećenje, male sisteme ili aplikacije kod kojih je rezerva važnija od troška. Njen glavni nedostatak je nemogućnost praćenja promjenjive potražnje: dimenzioniranje prema prosjeku povećava rizik zagušenja, a dimenzioniranje prema vrhu smanjuje iskorištenost.

Reaktivno skaliranje koristi trenutno ili nedavno izmjereno stanje. Prednosti su jednostavna interpretacija, manja zavisnost od historijskih podataka i sposobnost reagovanja na događaj koji ranije nije viđen. Nedostatak je kašnjenje između nastanka opterećenja, detekcije, odluke i aktiviranja resursa. Najbolje odgovara aplikacijama sa kratkim vremenom pokretanja i opterećenjem koje ne raste brže od upravljačke petlje.

Proaktivno skaliranje koristi prognozu i može pripremiti resurse prije vrha. Pogodno je za dnevne, sedmične ili sezonske obrasce. Zahtijeva kvalitetne podatke, redovno ponovno treniranje ili podešavanje i nadzor greške predviđanja. Strukturna promjena ponašanja korisnika može učiniti historijski model nepouzdanim upravo kada je reakcija najpotrebnija.

Hibridni pristup kombinuje prediktivni osnovni kapacitet sa reaktivnom korekcijom. Time ublažava kašnjenje predikcije i rizik neočekivanog vrha, ali povećava broj parametara i mogućnost konflikta. Uspješna primjena zahtijeva jasna pravila prioriteta, zajedničke granice i različite vremenske horizonte za pojedine kontrolere [7], [8], [19].

Kod schedulinga sličan kompromis postoji između centralizovanog i distribuiranog odlučivanja. Centralizovani scheduler ima širi pogled na stanje, ali može postati usko grlo. Omega uvodi paralelne schedulere koji dijele stanje i koriste optimističku kontrolu konkurentnosti, čime povećava skalabilnost uz potrebu rješavanja konflikata [15].

Politike usmjerene na pravičnost i politike usmjerene na rok također ne daju isti rezultat. Pravična raspodjela štiti korisnike od monopolizacije resursa, dok deadline pristup daje prednost zadacima sa vremenskim ograničenjem. U mješovitom sistemu potrebno je rezervisati kapacitet za kritične klase i istovremeno sprečavati glad ostalih zadataka.

Najpovoljniji pristup zato se bira prema profilu aplikacije. Interaktivna usluga sa strogim repnim vremenom odziva traži rezervu i brzu reaktivnu zaštitu. Periodični analitički posao može koristiti prediktivno pripremanje. Batch zadatak tolerantan na prekid može biti preemptiran i izvršavan na jeftinijem višku kapaciteta. Jedinstvena politika za sve tri klase bila bi jednostavnija, ali vjerovatno manje efikasna.

Rezultat poređenja je odluka u korist adaptivnog, višekriterijskog pristupa, ali ne nužno najsloženijeg mogućeg rješenja. Politika treba biti dovoljno složena da obuhvati važne karakteristike opterećenja, a dovoljno jednostavna da se može testirati, objasniti i sigurno održavati.

Tabela 6.2. Komparativna analiza pristupa upravljanju

Pristup	Reakcija	Složenost	Najpogodniji uslovi
Statički	Nema automatske promjene	Niska	Stabilno opterećenje i jaka rezerva
Reaktivni	Nakon izmjerenog odstupanja	Srednja	Brzo pokretanje i nepredvidivi događaji
Proaktivni	Prije očekivane promjene	Visoka	Ponavljajući i predvidivi obrasci
Hibridni	Prognoza uz brzu korekciju	Visoka	Mješavina obrazaca i neočekivanih vrhova

Izvor: izrada autora prema [7], [8], [19] i [23].

6.4. Razmatranje hipoteza istraživanja

Prva hipoteza pretpostavlja da kvalitetno upravljanje resursima i njihovom alokacijom može minimizirati troškove pružanja usluge u oblaku. Literatura o elastičnosti, pravilnom dimenzioniranju i troškovno svjesnom autoscalingu opisuje mehanizme smanjenja neiskorištenog kapaciteta [7]-[9]. Ilustrativni proračun u ovom radu pokazuje isti smjer, ali nije empirijsko poređenje politika.

H1 je na osnovu teorijske i ilustrativne analize podržana uslovno. Praktična evaluacija u sedmom poglavlju provjerava ovaj mehanizam poređenjem statičke i dinamičkih politika pri istom opterećenju, uz odvojeno izvještavanje aktiviranog kapaciteta i kvaliteta usluge.

Druga hipoteza pretpostavlja da kvalitetno upravljanje resursima i njihova alokacija mogu unaprijediti efikasnost upotrebe IT resursa. Nalazi o pakovanju zadataka, pravičnosti, lokalnosti podataka i dinamičkom skaliranju podržavaju mehanizme koji mogu povećati koristan rad po jedinici kapaciteta [10]-[16].

H2 je teorijski podržana uz uslov da se efikasnost mjeri skupom pokazatelja. Veća iskorištenost nije poboljšanje ako povećava greške ili neprihvatljivo pogoršava vrijeme obrta.

Sedmo poglavlje zato zajedno posmatra kapacitet, protok, čekanje, vrijeme obrta i stopu grešaka.

Ocjene u ovom poglavlju predstavljaju teorijsku osnovu za praktičnu evaluaciju. Konačna eksperimentalna ocjena hipoteza izvedena je u sedmom poglavlju i formulirana je uslovno, u granicama ispitane aplikacije i lokalnog Kubernetes okruženja.

Tabela 6.3. Teorijska ocjena istraživačkih hipoteza

Hipoteza	Teorijska ocjena	Potrebna empirijska provjera
H1: smanjenje troškova	Uslovno podržana	Worker-minute, cijena kapaciteta i kvalitet usluge za S0-S2
H2: povećanje efikasnosti	Uslovno podržana	Protok, čekanje, obrt, greške i iskorištenost za S0-S2

Izvor: izrada autora na osnovu literature i ilustrativnog proračuna.

6.5. Preporuke za efikasnije upravljanje resursima u oblaku

Prva preporuka je definisati mjerljive ciljeve prije izbora algoritma. Potrebno je odrediti prihvatljivo vrijeme odziva, raspoloživost, rokove, budžet i granice potrošnje energije. Bez jasnog cilja nije moguće ocijeniti da li je veća iskorištenost stvarno poboljšanje.

Druga preporuka je povezati infrastrukturnu i aplikacijsku telemetriju. Procesor i memorija opisuju stanje resursa, dok red čekanja, stopa zahtjeva, percentili vremena odziva i greške opisuju korisnički učinak. Odluka o skaliranju treba koristiti metriku koja je uzročno povezana sa problemom.

Treća preporuka je koristiti histerezu, periode stabilizacije i ograničenja brzine promjene. Ovi mehanizmi sprečavaju oscilovanje i daju sistemu vrijeme da pokaže učinak prethodne akcije. Minimalni i maksimalni kapacitet moraju štititi i budžet i dostupnost.

Četvrta preporuka je razdvojiti politike prema klasi opterećenja. Interaktivne usluge, batch poslovi, baze podataka i workflow aplikacije imaju različite zahtjeve. Prioritet, mogućnost preempcije, prihvatljivo kašnjenje i model skaliranja treba prilagoditi svakoj klasi.

Peta preporuka je kombinovati reaktivno i proaktivno upravljanje kada opterećenje sadrži i ponavljajuće obrasce i neočekivane skokove. Predikcija može osigurati osnovni kapacitet, dok

reaktivna zaštita preuzima odstupanja. Kontroleri moraju imati jasno definisane nadležnosti kako ne bi donosili suprotne odluke.

Šesta preporuka je redovno provoditi analizu pravilnog dimenzioniranja i uklanjati napuštene resurse. Rezervisani kapacitet, skladište, mrežni prenos i licence treba posmatrati zajedno. Tehničke metrike potrebno je povezati sa vlasnikom i poslovnom vrijednošću usluge.

Sedma preporuka je testirati politiku na reprezentativnim tragovima prije široke primjene. Korisno je provesti shadow ili recommendation režim u kojem sistem bilježi predložene akcije bez izvršavanja, zatim kontrolisani pilot i tek onda automatsku primjenu. Svaka promjena treba imati plan povratka.

Osma preporuka je uključiti energiju i održivost u isti proces odlučivanja. Konsolidacija, gašenje praznih čvorova i data-aware scheduling često istovremeno smanjuju trošak i potrošnju. Metrike infrastrukture treba dopuniti mjerom korisnog rada po jedinici energije.

Konačno, upravljanje resursima treba posmatrati kao kontinuiran proces. Promjena aplikacije, cijena, radnog opterećenja ili platforme mijenja i optimalnu politiku. Periodična evaluacija osigurava da konfiguracija koja je nekada bila dobra ne postane izvor trajne neefikasnosti.

7. Praktična evaluacija politika alokacije resursa

Poglavlje prikazuje lokalni pilot i konačnu Kubernetes evaluaciju provedenu kroz 60 strogo validiranih izvođenja. Rezultati se tumače deskriptivno i isključivo unutar istog profila opterećenja.

7.1. Cilj evaluacije i istraživačka pitanja

Cilj praktične evaluacije je eksperimentalno usporediti statičku alokaciju workera, reaktivno horizontalno skaliranje prema iskorištenosti procesora i skaliranje prema aplikacijskoj metrici reda poslova. Poređenje se provodi pri istim verzijama aplikacije, ulaznim profilima, parametrima obrade i ograničenjima resursa. Time se ispituje kako politika alokacije utiče na aktivirani kapacitet, iskorištenost resursa, protok i vrijeme obrta.

Evaluacija odgovara na sljedeća pitanja:

1. Da li automatsko skaliranje smanjuje broj aktivnih worker-minuta u odnosu na statičku alokaciju pri istom profilu opterećenja?
2. Kako politika skaliranja utiče na prosječno, medijalno i P95 vrijeme obrta, čekanje u redu i stopu neuspješnih poslova?
3. Koliko vrijeme pokretanja novih replika utiče na reakciju sistema na postepene i nagle promjene opterećenja?
4. Postoji li konfiguracija koja smanjuje aktivirani kapacitet bez neprihvatljivog pogoršanja kvaliteta usluge?

7.2. Sistem i eksperimentalno okruženje

Cloud Media Processor je kontejnerizovana aplikacija za asinhronu batch obradu slika. Web i API sloj prihvataju zahtjeve, ulazne datoteke se čuvaju u MinIO objektnoj pohrani, a BullMQ i Redis upravljaju redom poslova. Worker procesi preuzimaju poslove i izvršavaju transformacije slika. Arhitektura omogućava promjenu broja worker replika bez promjene API-ja, podataka ili vrste posla.

Eksperimentalna telemetrija obuhvata dužinu reda, broj aktivnih poslova i workera, završene i neuspješne poslove, trajanje obrade i čekanja, CPU i RSS memoriju worker procesa te alocirane CPU i memorijske requeste. Svaki posao i periodični uzorak vezani su za jedinstvenu eksperimentalnu sesiju, čime se sprečava miješanje podataka između izvođenja.

7.2.1. Konačno Kubernetes okruženje

Konačna matrica izvedena je 10. oktobra 2026. na jednom lokalnom `kind` čvoru u Docker Desktop virtuelnoj mašini. Host je MacBook Pro s Apple M1 Pro procesorom, osam CPU jezgri i 16 GB fizičke memorije. Docker Desktop virtuelnoj mašini bilo je dostupno osam CPU jezgri i približno 8 GB memorije. Generator opterećenja k6 izvršavao se na hostu i pristupao aplikacijskom API-ju preko lokalnog NodePorta `30080`.

Eksperimentalni klaster koristio je `kind` 0.33.0, Kubernetes server 1.37.0 za `linux/arm64`, jedan control-plane čvor, Debian GNU/Linux 13 i `containerd` 2.3.4. Metrics Server je bio verzije 0.9.0, a KEDA 2.21.0. Runner je koristio Node.js 22.18.0 i k6 2.3.0 za `darwin/arm64`. Kubernetes klijent bio je verzije 1.32.2; zabilježena razlika između klijentske i serverske minor verzije nije uticala na završetak mjerenja, ali je navedena među prijetnjama reproduktivnosti.

Radni kontejner imao je CPU request od 500 mCPU i limit od jedne CPU jezgre, te memorijski request od 256 MiB i limit od 512 MiB. API je koristio 100 mCPU/128 MiB request i 500 mCPU/256 MiB limit, MinIO 100 mCPU/192 MiB i 500 mCPU/512 MiB, Redis 50 mCPU/64 MiB i 250 mCPU/192 MiB, a web komponenta 50 mCPU/64 MiB i 250 mCPU/128 MiB. Sve politike koristile su istu aplikacijsku sliku servisa i workera, `sha256:0481b0757ae11471eb3dc8305f00873cb1fd1dc204d8484d2de456ae9d3ce8f0`, i istu web sliku, `sha256:e69da37214c83b4a93ce8783d3e5f61ff06ed9b3626879e8b734ddcc62d4d32c`.

7.3. Eksperimentalne politike i profili opterećenja

Jedina namjerno promijenjena nezavisna varijabla je politika alokacije worker replika:

Tabela 7.1. Eksperimentalne politike alokacije worker replika

Oznaka	Politika	Konfiguracija
S0	STATIC	šest replika bez automatskog skaliranja
S1	CPU_HPA	jedna do šest replika, cilj 65% iskorištenosti CPU requesta
S2	QUEUE_KEDA	jedna do šest replika, cilj pet poslova na čekanju po replici

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.

Svaka politika izvršava četiri profila opterećenja:

Tabela 7.2. Profili eksperimentalnog opterećenja

Profil	Osnovni oblik	Namjena
low	1 zahtjev/s tokom 5 min	provjera prekomjerne alokacije pri malom stabilnom opterećenju
high	6 zahtjeva/s tokom 5 min	ponašanje pri trajnom visokom opterećenju
ramp	1 -> 8 -> 2 zahtjeva/s	reakcija na postepeni rast i pad
spike	1 -> 15 -> 1 zahtjeva/s	reakcija i oporavak nakon naglog vrha

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.

Svaka od 12 kombinacija politike i profila ponavlja se pet puta. Ukupno se izvodi 60 mjerenja. Deterministčan uravnotežen raspored rotira početnu politiku i redoslijed profila, čime se smanjuje sistematski uticaj redoslijeda izvođenja.

7.4. Metrike i veza sa hipotezama

Prva hipoteza (H1) odnosi se na smanjenje troška pružanja usluge. Operacionalizuje se brojem worker-minuta, CPU-request-minutama, udjelom neiskorištenog kapaciteta i, samo uz dokumentovanu jediničnu cijenu, procijenjenim troškom alociranog kapaciteta. Ako se eksperiment izvodi na fiksno naplaćenim serverima, rezultat se ne predstavlja kao stvarno smanjenje cloud računa.

Druga hipoteza (H2) odnosi se na unapređenje efikasnosti upotrebe IT resursa. Operacionalizuje se brojem završenih poslova u minuti, vremenom obrade, čekanja i obrta, CPU i memorijskom iskorištenošću, stopom grešaka te vremenom reakcije skaliranja.

CPU iskorištenost alociranog kapaciteta računa se u odnosu na CPU request, a ne na limit. Zato vrijednost može biti veća od 100% kada proces koristi raspoloživi kapacitet iznad rezervisanog requesta.

Za svaku kombinaciju politike i profila izračunate su srednja vrijednost, medijan, uzoračka standardna devijacija, minimum, maksimum i P95 između pet ponavljanja. Relativna promjena dinamičke politike računa se u odnosu na srednju vrijednost statičke politike samo unutar istog profila. Zbog malog broja ponavljanja i unaprijed definisanog deskriptivnog plana ne izvode se naknadni testovi statističke značajnosti. Zaključci se zasnivaju na veličini, smjeru i ponovljivosti opaženih razlika, bez generalizacije izvan ispitanog sistema.

7.5. Postupak i kontrola valjanosti

Automatizovani runner prije svakog izvođenja provjerava dostupnost aplikacije i potrebnih alata, prazno stanje reda, odsustvo druge aktivne sesije, aktivnu politiku i početni broj replika. Nakon pokretanja jedinstvene sesije, k6 šalje poslove prema zadanom arrival-rate profilu. Završetak se ne prati statusnim anketiranjem svakog posla, jer bi takvi zahtjevi kontaminirali opterećenje API-ja i Redis-a.

Po završetku generisanja opterećenja runner čeka da red, broj aktivnih i broj nedovršenih poslova dostignu nulu. Zatim arhivira JSON i CSV sesije, k6 sažetak, vremensku seriju, stanje skalera i relevantne logove. Izvođenje je nevalidno ako postoji neuspješan ili nedovršen posao, odbačena k6 iteracija, neusklađena politika, pogrešan početni broj workera ili nepotpun artefakt.

Konačna analiza prihvata samo skup sa pet tačno označenih ponavljanja svake kombinacije, bez duplikata i isključenih izvođenja, uz jednu Git reviziju i iste image digest vrijednosti.

7.6. Lokalni pilot-test

7.6.1. Svrha i ograničenja

Lokalni pilot izveden je 7. oktobra 2026. u Docker Desktop okruženju s politikom `STATIC` i šest worker kontejnera. Njegova svrha bila je provjeriti mjerni lanac i potvrditi da profili `low` i `spike` proizvode dovoljno različita stanja sistema. Radno stablo tokom pilota nije bilo čisto jer su eksperimentalne skripte aktivno provjeravane. Zbog toga pilot nije dio konačnog eksperimentalnog skupa.

7.6.2. Rezultati pilota

Tabela 7.3. Rezultati lokalnog pilot-testa

Metrika	low	spike
poslani / završeni poslovi	301 / 301	1260 / 1260
neuspješni / nedovršeni poslovi	0 / 0	0 / 0
maksimalna dužina reda	0	271
prosječna dužina reda	0	64,11
prosječno vrijeme obrade	496,24 ms	548,89 ms
P95 vrijeme obrade	511 ms	618 ms
prosječno vrijeme obrta	498,97 ms	11.991,22 ms
P95 vrijeme obrta	513 ms	25.294 ms
prosječno čekanje u redu	1,12 ms	11.440,58 ms

Metrika	low	spike
maksimalno čekanje u redu	6 ms	25.143 ms
prosječan throughput	60,06 poslova/min	356,38 poslova/min
worker-minute	30,07	21,21
poslovi po worker-minuti	10,01	59,40
prosječna iskorištenost CPU requesta	20,62%	113,20%
procijenjeni neiskorišteni kapacitet	79,38%	0%

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.

7.6.3. Tumačenje pilota

Profil `low` nije formirao red i pokazao je izraženu prekomjernu alokaciju statičkog kapaciteta. Nasuprot tome, `spike` je formirao red od 271 posla, ali je sistem završio svih 1260 poslova bez greške i potpuno ispraznio red.

Prosječno vrijeme obrade poraslo je samo 10,61%, sa 496,24 ms na 548,89 ms. Istovremeno je prosječno vrijeme obrta poraslo 24,03 puta, a P95 vrijeme obrta 49,31 puta. Prosječno čekanje u redu od 11,44 s čini približno 95,4% prosječnog vremena obrta u `spike` profilu. Nalaz pokazuje da glavni uzrok degradacije nije značajno sporija obrada pojedinačnog posla, nego privremeni nedostatak paralelnog kapaciteta.

Veći throughput i broj poslova po worker-minuti u `spike` profilu nisu dokaz bolje politike, jer oba profila koriste istih šest statičkih workera, a ulazno opterećenje i trajanje izvođenja nisu jednaki. Slično tome, niži zbir worker-minuta u `spike` profilu posljedica je kraćeg ukupnog trajanja tog izvođenja i ne smije se tumačiti kao ušteda.

Prvo `spike` izvođenje otkrilo je metodološku grešku: k6 je anketirao status svakog batcha svakih 200 ms, proizveo 50.100 dodatnih HTTP zahtjeva i odbacio 142 planirane iteracije. To izvođenje je isključeno. Nakon razdvajanja generisanja opterećenja od mjerenja završetka, ponovljeni `spike` poslao je svih 1260 planiranih poslova bez odbačenih iteracija. Pilot je time ispunio svoju glavnu svrhu - otkrio je i uklonio izvor sistematske mjerne pristrasnosti prije konačnog eksperimenta.

7.6.4. Status hipoteza nakon pilota

Pilot ne omogućava prihvatanje ni odbacivanje H1 ili H2. Izvedena je samo statička politika, jedinična cijena worker-kapaciteta nije definisana, a svaki profil ima samo jedno validno ponavljanje. Pilot potvrđuje osjetljivost mjernog sistema na prekomjernu i nedovoljnu alokaciju, ali ne utvrđuje učinak automatskog skaliranja.

7.7. Konačni rezultati

Konačni rezultati potiču iz potpune matrice od tri politike, četiri profila i pet ponavljanja. Srednja vrijednost i uzoračka standardna devijacija u tabelama se zapisuju kao `srednja vrijednost ± SD`.

7.7.1. Potpunost skupa podataka

Stroga analiza uključila je svih 60 izvođenja i nije isključila nijedno. Svaka od 12 kombinacija ima tačno pet ponavljanja, bez duplikata i nedostajućih rednih brojeva. U svim izvođenjima poslano je i završeno 67.954 posla, bez neuspješnih i nedovršenih poslova. Generator opterećenja nije zabilježio odbačene iteracije, a svi obavezni artefakti su sačuvani.

Cijela matrica koristi čistu Git reviziju `3a599c6d45cbf0947171150050ebbb00b347f5b6` i iste immutable digest vrijednosti aplikacijskih slika. Time su ispunjeni unaprijed definisani kriteriji uključivanja u konačnu analizu.

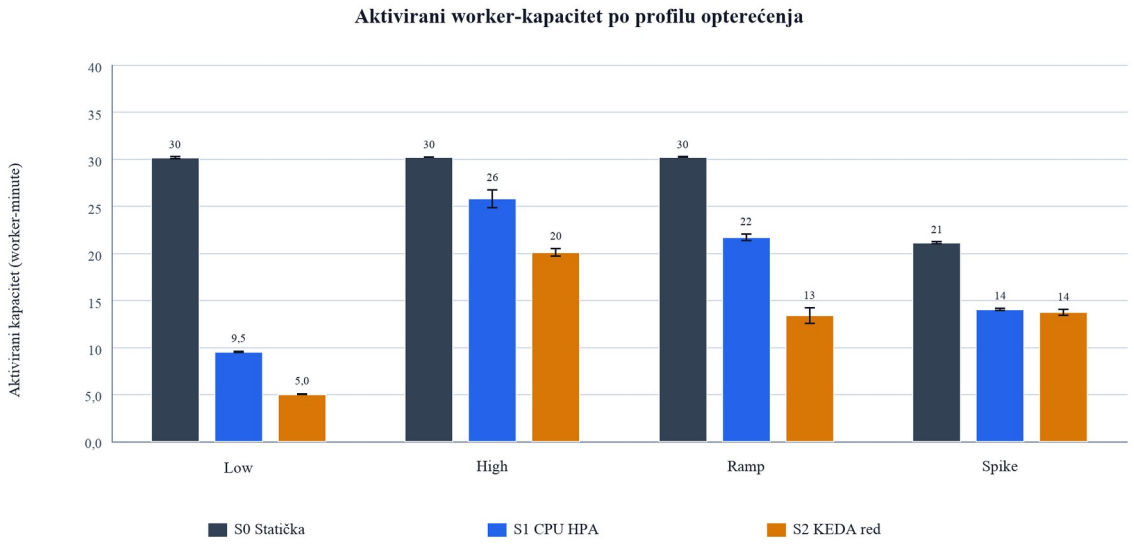
7.7.2. Aktivirani kapacitet

Tabela 7.4 prikazuje aktivirani worker-kapacitet. Obje dinamičke politike smanjile su worker-minute u sva četiri profila. Najveće smanjenje ostvareno je pri malom opterećenju: CPU HPA za 68,4%, a KEDA za 83,4% u odnosu na statičkih 30,14 worker-minuta. KEDA je zadržala jednu repliku i ostvarila 5,02 worker-minute, dok je CPU HPA u prosjeku koristio 1,89 replika i 9,51 worker-minutu.

Tabela 7.4. Aktivirani kapacitet i efikasnost workera

Profil	Politika	Worker-minute, sredina ± SD	Promjena prema S0	Prosječno workera	Poslova po worker-minuti
low	S0 statička	30,14 ± 0,11	-	6,00	9,97
low	S1 CPU HPA	9,51 ± 0,06	-68,4%	1,89	31,60
low	S2 KEDA red	5,02 ± 0,02	-83,4%	1,00	59,89
high	S0 statička	30,22 ± 0,01	-	6,00	59,60
high	S1 CPU HPA	25,77 ± 0,95	-14,7%	5,11	69,95
high	S2 KEDA red	20,10 ± 0,39	-33,5%	3,99	89,61
ramp	S0 statička	30,22 ± 0,01	-	6,00	38,71
ramp	S1 CPU HPA	21,68 ± 0,35	-28,3%	4,32	53,95
ramp	S2 KEDA red	13,38 ± 0,82	-55,7%	2,59	87,62
spike	S0 statička	21,10 ± 0,11	-	6,00	59,69
spike	S1 CPU HPA	14,03 ± 0,12	-33,5%	4,04	89,76
spike	S2 KEDA red	13,73 ± 0,32	-34,9%	3,94	91,74

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.



Slika 7.1. Aktivirani worker-kapacitet po profilu opterećenja

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.

Promjene u CPU-request-minutama jednake su relativnim promjenama worker-minuta jer svaka replika ima isti request od 0,5 CPU jezgre. Jedinična novčana cijena worker-minute nije postavljena. Rezultati zato opisuju smanjenje aktiviranog aplikacijskog kapaciteta, a ne stvarno smanjenje cloud računa.

7.7.3. Kvalitet usluge

Sva tri pristupa zadržala su jednak broj završenih poslova i nultu stopu grešaka. Prosječni throughput unutar istog profila bio je gotovo jednak, što pokazuje da dinamičke politike nisu izgubile posao niti smanjile ukupnu količinu rada. Razlike se pojavljuju u čekanju i vremenu obrta.

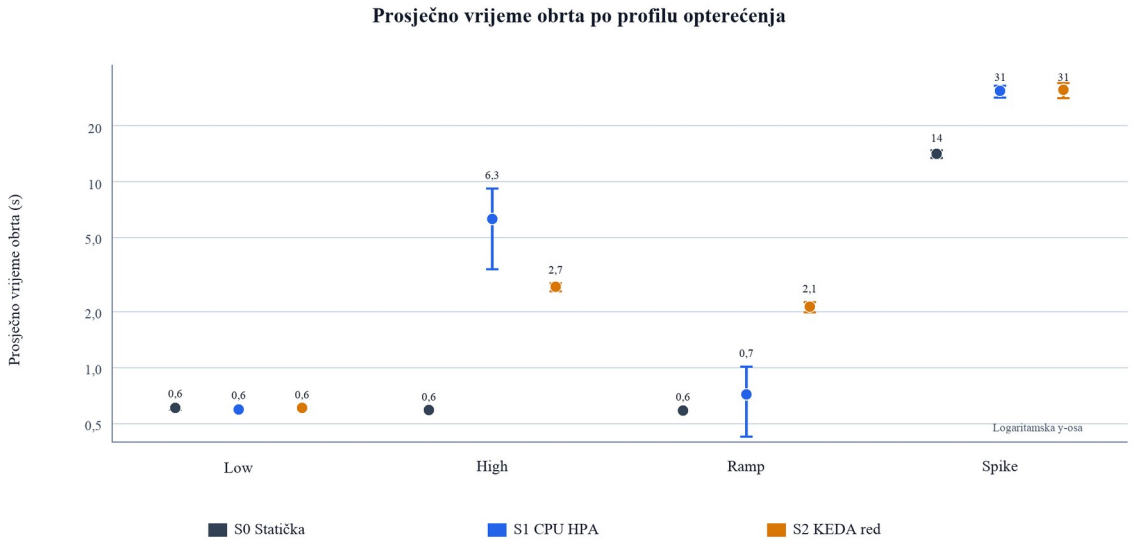
Tabela 7.5. Vrijeme obrta, čekanje i throughput

Profil	Politika	Prosječni obrt, ms ± SD	P95 obrt, ms	Prosječno čekanje, ms	Throughput, poslova/min
low	S0 statička	605,30 ± 10,20	628,80	1,20	59,85
low	S1 CPU HPA	596,52 ± 6,89	612,60	1,18	59,84
low	S2 KEDA red	604,86 ± 3,45	617,60	1,24	59,89
high	S0 statička	588,66 ± 2,77	607,20	0,54	357,60
high	S1 CPU HPA	6.286,10 ± 2.901,20	24.039,20	5.694,24	357,54
high	S2 KEDA red	2.709,85 ± 128,27	9.638,60	2.118,21	357,43

Tabela 7.5. Vrijeme obrta, čekanje i throughput (nastavak)

Profil	Politika	Prosječni obrt, ms ± SD	P95 obrt, ms	Prosječno čekanje, ms	Throughput, poslova/min
ramp	S0 statička	587,48 ± 3,30	614,00	0,53	232,23
ramp	S1 CPU HPA	718,80 ± 293,40	1.409,60	110,39	232,14
ramp	S2 KEDA red	2.123,31 ± 135,00	6.405,60	1.533,65	231,54
spike	S0 statička	14.090,39 ± 623,65	29.436,60	13.507,42	358,17
spike	S1 CPU HPA	30.586,96 ± 2.211,46	48.789,80	29.997,47	357,46
spike	S2 KEDA red	31.012,22 ± 2.886,35	48.733,80	30.424,72	358,20

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.



Slika 7.2. Prosječno vrijeme obrta po profilu opterećenja

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.

Pri profilu `low` smanjenje kapaciteta nije pogoršalo vrijeme obrta. Prosječne vrijednosti sve tri politike ostale su između 596,52 i 605,30 ms. Pri trajnom profilu `high`, statičkih šest replika spriječilo je red, dok su dinamičke politike prvo morale povećati kapacitet. CPU HPA povećao je prosječno vrijeme obrta za 967,9%, a KEDA za 360,3% prema S0. KEDA je u ovom profilu ostvarila povoljniji kompromis od CPU HPA: koristila je 33,5% manje worker-minuta od S0, uz prosječni obrt od 2,71 s, dok je CPU HPA uštedio 14,7% uz 6,29 s.

U profilu `ramp` CPU HPA je u četiri od pet izvođenja zadržao prosječni obrt između 584,69 i 591,55 ms, dok je jedno izvođenje dostiglo 1.243,63 ms. Zbog toga srednja vrijednost iznosi 718,80 ms uz veću standardnu devijaciju. KEDA je agresivnije smanjila kapacitet, ali je redovno formirala red i ostvarila prosječni obrt od 2,12 s.

Profil `spike` premašio je kratkoročni kapacitet i statičke konfiguracije. Statička politika imala je prosječni obrt 14,09 s, a dinamičke politike približno 31 s. Obje dinamičke politike započele su sa jednom replikom i dostigle maksimalnih šest tek nakon nastanka reda, pa je ušteda worker-minuta ostvarena uz približno dvostruko duže prosječno vrijeme obrta.

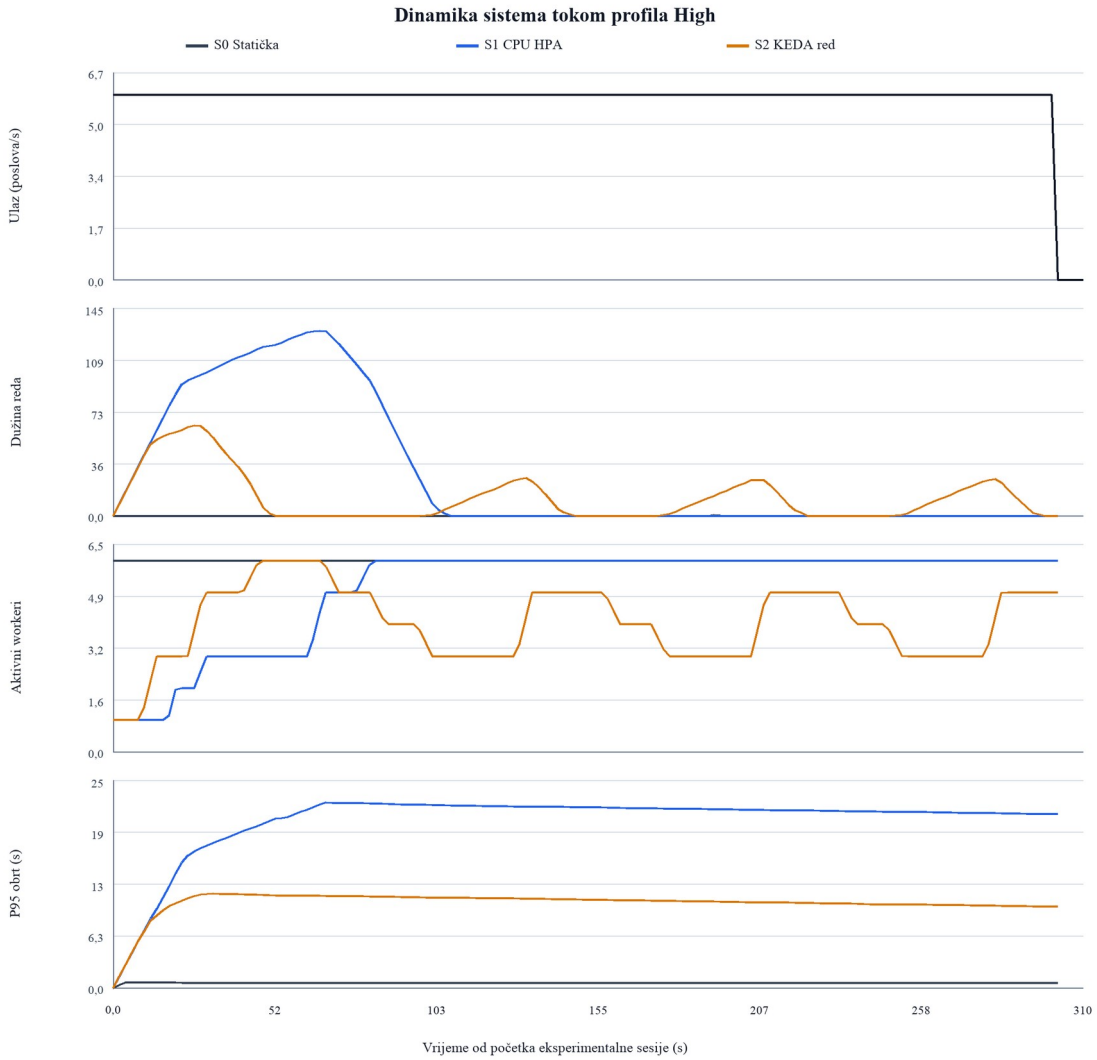
7.7.4. Dinamika skaliranja

Vremenski prikaz koristi medijan pet ponavljanja. P95 vrijeme obrta u svakom trenutku ponovo je izračunato samo iz job događaja konkretne eksperimentalne sesije, čime se sprečava prenošenje kumulativnih vrijednosti između uzastopnih izvođenja istog namespacea.

Tabela 7.6. Dinamika automatskog skaliranja

Profil	Politika	Reakcija scale-upa, s ± SD	Akcije skaliranja	Oscilacije	Maksimalno workera
low	S1 CPU HPA	31,89 ± 1,49	1,00	0,00	2,00
low	S2 KEDA red	nije aktivirano	0,00	0,00	1,00
high	S1 CPU HPA	29,88 ± 13,02	3,00	0,00	6,00
high	S2 KEDA red	14,79 ± 1,85	12,80	6,20	6,00
ramp	S1 CPU HPA	27,86 ± 2,94	3,60	0,00	6,00
ramp	S2 KEDA red	101,73 ± 4,10	10,20	5,00	6,00
spike	S1 CPU HPA	38,59 ± 5,48	3,20	0,00	6,00
spike	S2 KEDA red	75,27 ± 3,86	3,00	0,00	6,00

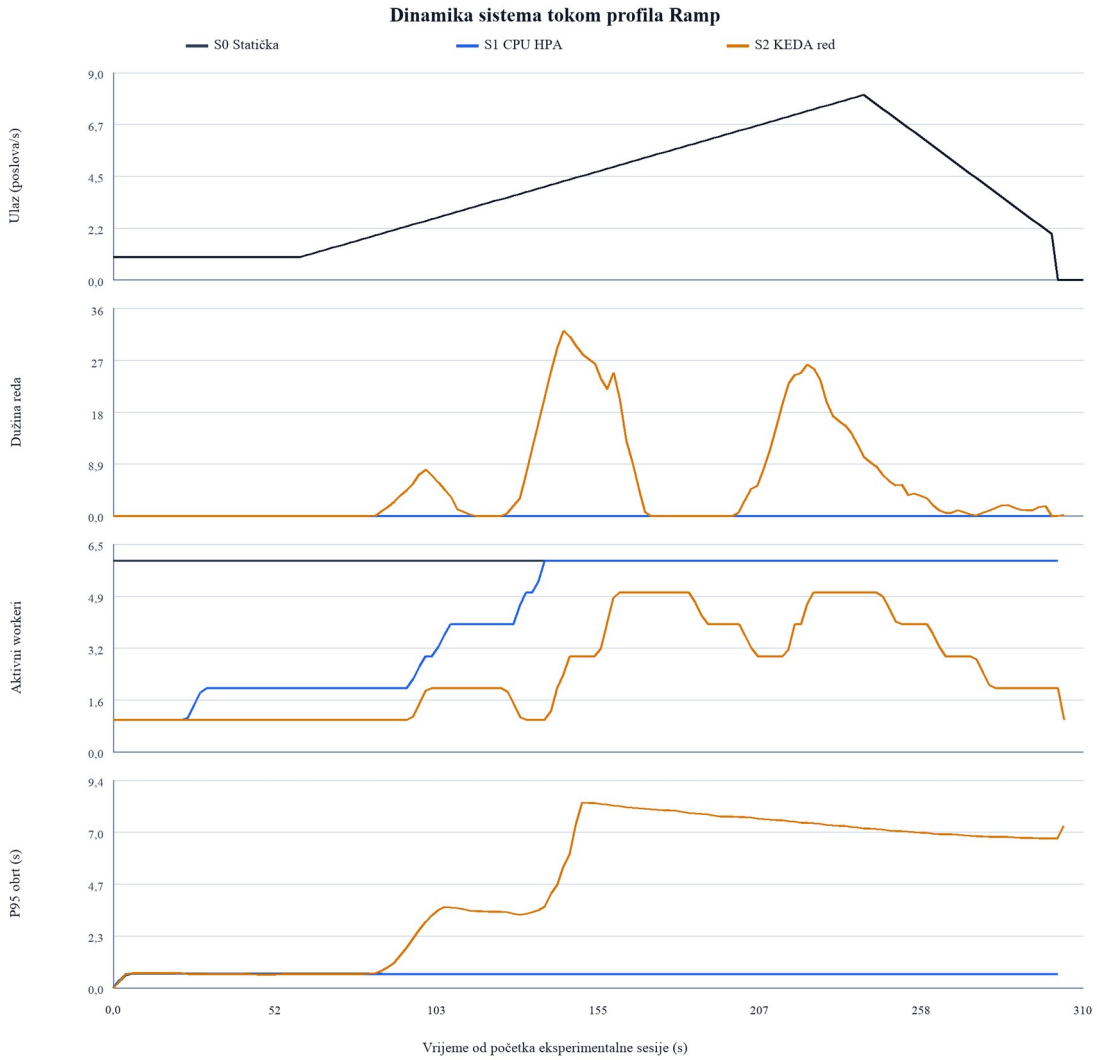
Izvor: izrada autora na osnovu eksperimentalnih mjerenja.



Slika 7.3. Dinamika sistema tokom profila high

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.

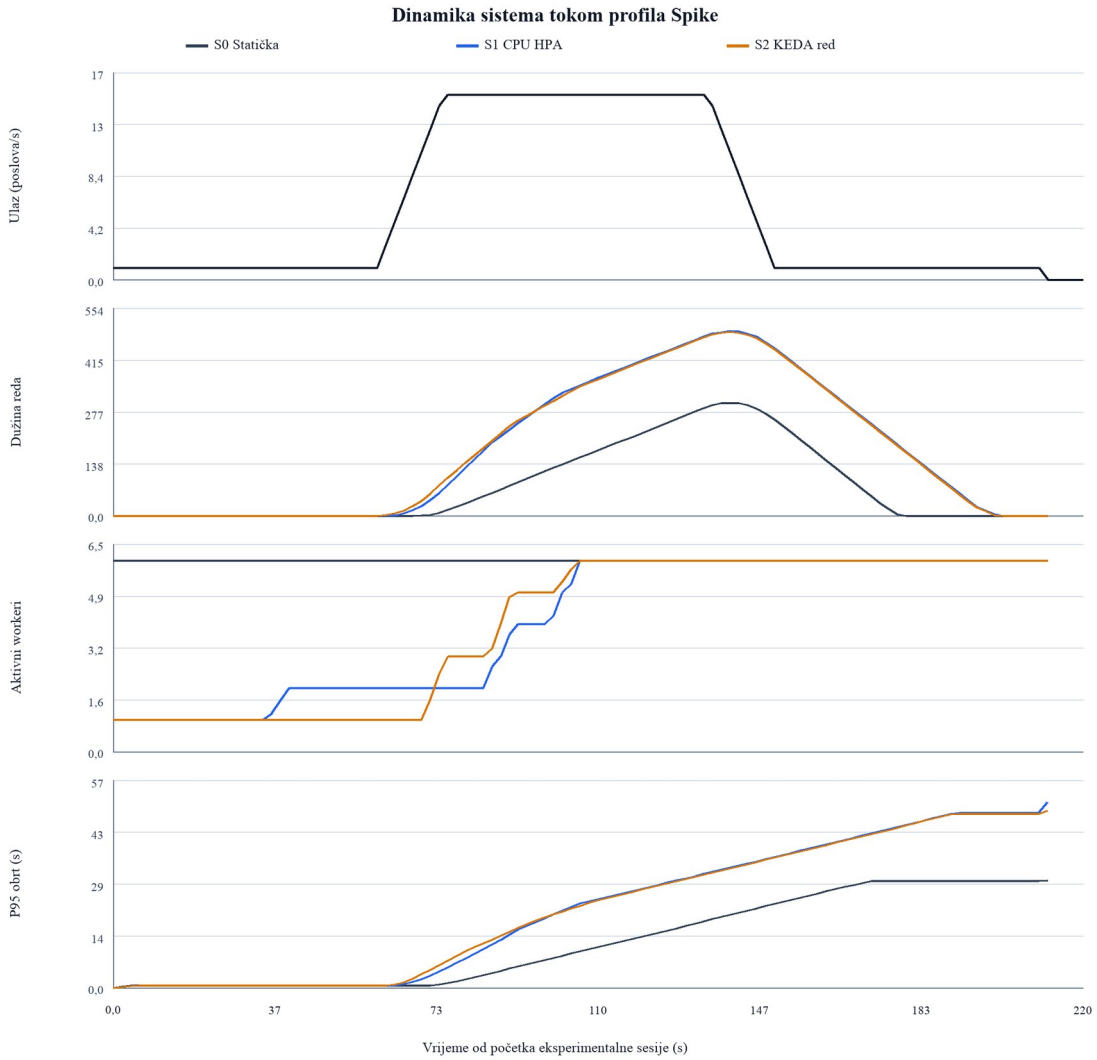
KEDA je u profilu `high` reagovala približno dvostruko brže od CPU HPA, ali je poslije početnog povećanja oscilovala između tri i šest replika. Ta dinamika objašnjava 6,2 prosječne oscilacije i 12,8 akcija. U profilu `ramp` KEDA-in redni okidač nije reagovao dok jedna replika nije prestala pratiti rast ulaza; zbog toga je izmjerena reakcija od 101,73 s. CPU HPA je reagovao na porast procesorske iskorištenosti prije formiranja značajnog reda i u četiri izvođenja zadržao gotovo statičko vrijeme obrta.



Slika 7.4. Dinamika sistema tokom profila ramp

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.

U profilu `spike` oba skalera dostigla su šest replika, ali tek nakon što je red već brzo rastao. Vrijeme reakcije zato nije dovoljno samo za ocjenu: CPU HPA je ranije dodao drugu repliku, a KEDA brže povećala broj replika u srednjem dijelu rasta, ali su završni P95 i prosječno vrijeme obrta ostali gotovo jednaki.



Slika 7.5. Dinamika sistema tokom profila spike

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.

7.7.5. Odnos kapaciteta i vremena obrta

Rezultati pokazuju Pareto-kompromis. Statička politika daje najnižu latenciju kada je svih šest replika potrebno odmah, ali pri `low` profilu troši šest puta više worker-kapaciteta od KEDA-e. KEDA najviše smanjuje worker-minute u sva četiri profila, ali redni okidač dopušta formiranje reda prije povećanja kapaciteta. CPU HPA je konzervativniji: troši više kapaciteta od KEDA-e, ali je u profilu `ramp` ostvario znatno niže vrijeme obrta.

Bez unaprijed definisanog SLO praga nije metodološki opravdano naknadno označiti određeno pogoršanje kao prihvatljivo. Konačan izbor politike zato zavisi od toga da li sistem prvenstveno optimizira aktivirani kapacitet ili repnu latenciju. Za strogi latency SLO potrebna je veća minimalna replika ili proaktivno skaliranje prije predvidivog vrha.

7.8. Ocjena hipoteza

H1 je eksperimentalno podržana u dijelu koji se odnosi na smanjenje aktiviranog aplikacijskog kapaciteta. Obje dinamičke politike smanjile su worker-minute u svakom profilu, a relativno smanjenje kretalo se od 14,7% do 83,4%. KEDA je u sva četiri profila ostvarila najniži prosječni aktivirani kapacitet. Hipoteza nije potvrđena kao smanjenje stvarnog novčanog računa jer je eksperiment izveden na fiksno alociranom lokalnom hostu i nije korištena dokumentovana jedinična cijena. Nalaz zato podržava mehanizam potencijalnog smanjenja troška, ne stvarno ostvarenu finansijsku uštedu.

H2 je podržana uslovno i parcijalno. Dinamičke politike povećale su broj poslova po worker-minuti između 17,4% i 500,4%, zadržale gotovo isti throughput i završile sve poslove bez greške. Time je poboljšana efikasnost korištenja aktiviranih worker resursa. Ipak, pri profilima `high`, `ramp` i `spike` manji početni kapacitet povećao je čekanje i vrijeme obrta. KEDA je najefikasnija prema worker-minutama, CPU HPA pruža bolji kompromis pri postepenom rastu, a statička politika štiti latenciju kada se vršni kapacitet mora obezbijediti odmah. Rezultati zato ne podržavaju tvrdnju da autoscaling bezuslovno poboljšava ukupan kvalitet usluge.

Tabela 7.7. Eksperimentalna ocjena istraživačkih hipoteza

Hipoteza	Eksperimentalna ocjena	Obrazloženje
H1	parcijalno podržana	Dinamičke politike smanjile su worker-minute u svim profilima, ali nije mjerena stvarna novčana ušteda.
H2	uslovno i parcijalno podržana	Efikasnost aktiviranih workera je povećana bez gubitka poslova, ali su zahtjevniji profili imali veće čekanje i vrijeme obrta.

Izvor: izrada autora na osnovu eksperimentalnih mjerenja.

7.9. Prijetnje valjanosti

Internu valjanost mogu ugroziti razlike u zagrijavanju kontejnera, keširanju imagea, dijeljenom fizičkom hardveru, stanju MinIO i Redis servisa te redoslijedu izvođenja. Rizik se smanjuje jednakim konfiguracijama, uravnoteženim rasporedom, periodom hlađenja, provjerom praznog reda i ponavljanjem mjerenja.

Konstruktna valjanost ograničena je činjenicom da CPU i RSS worker procesa ne obuhvataju kompletnu potrošnju Kubernetes noda. Worker-minute predstavljaju aktivirani aplikacijski kapacitet, a ne automatski stvarni račun javnog oblaka.

Eksterna valjanost ograničena je jednom aplikacijom, sintetičkim slikama, jednim lokalnim Kubernetes čvorom i kontrolisanim profilima. Klaster nema mrežnu latenciju ni scheduling karakteristike višestrukih fizičkih čvorova. Nalazi se zato ne generalizuju na sve cloud aplikacije, nego pokazuju ponašanje posmatrane arhitekture u dokumentovanim uslovima.

Pet ponavljanja po kombinaciji omogućava procjenu sredine i varijabilnosti, ali je uzorak mali za pouzdanu inferencijalnu generalizaciju. Analiza je zbog toga deskriptivna. Dodatno, nije unaprijed zadan SLO za vrijeme obrta, pa se kvalitet usluge prikazuje kao izmjereni kompromis bez naknadno postavljene granice prihvatljivosti.

Kubernetes klijent 1.32.2 bio je stariji od serverske verzije 1.37.0 izvan preporučenog minor version skew raspona. Eksperimentalne aplikacije, skaleri i telemetrija izvršavali su se na serveru, a sva mjerenja i kontrole završile su uspješno. Ipak, ponavljanje treba provesti s usklađenom klijentskom verzijom kako bi se uklonio ovaj reproduktivni rizik.

7.10. Reproduktivnost

Plan matrice generiše se komandom `EXPECTED_REPETITIONS=5 pnpm run:matrix`, pri čemu se finalni runovi čuvaju u `results/final`, odvojeno od pilot-podataka. Konačna agregacija pokreće se sa `RESULTS_DIR=results/final ANALYSIS_DIR=analysis/final STRICT_ANALYSIS=1 EXPECTED_REPETITIONS=5 pnpm analyze:experiments`. Strogi režim prekida analizu ako nedostaje ponavljanje, postoji duplikat, isključeno izvođenje, nedostajući artefakt ili više Git revizija.

Za svako izvođenje arhivirani su manifest, potpuni izvoz eksperimentalne sesije, CSV vremenska serija, k6 sažetak i log, Kubernetes stanje prije i poslije mjerenja, autoscaler stanje i relevantni logovi. Manifesti povezuju sirove podatke sa Git revizijom runnera, dok su skripte završne agregacije i grafikona sačuvane uz projekat. Time je moguće ponoviti agregaciju iz sirovih artefakata bez oslanjanja na vrijednosti prepisane u dokument.

ZAKLJUČAK

Upravljanje resursima i njihova alokacija predstavljaju jezgro cloud computinga. Elastičnost ostvaruje vrijednost tek kada sistem pravovremeno odredi potreban kapacitet, rasporedi zadatke i oslobodi resurse. Zbog promjenjivog opterećenja ovaj problem je višekriterijski i dinamičan.

Tehničke odluke ne mogu se zasnivati na jednoj metrici. CPU, memorija, mreža i skladište moraju se tumačiti zajedno sa protokom, čekanjem, vremenom obrta, greškama i aktiviranim kapacitetom. Visoka iskorištenost nije poboljšanje ako narušava pouzdanost ili kvalitet usluge.

Analiza scheduling pristupa pokazala je različite kompromise. Preempcija poboljšava odziv hitnih zadataka uz trošak prekida, prioriteti zahtijevaju zaštitu od gladi, a data-aware scheduling smanjuje mrežni trošak približavanjem izvršavanja podacima. Politika se zato bira prema profilu radnog opterećenja.

Dinamičko skaliranje može smanjiti razliku između aktiviranog i potrebnog kapaciteta. Reaktivni pristup reaguje nakon promjene, proaktivni priprema kapacitet prema prognozi, a hibridni pristup kombinuje oba mehanizma uz veću složenost. Stabilnost zahtijeva histerezu, periode stabilizacije i jasne granice.

Lokalni pilot potvrdio je osjetljivost mjernog postupka, a konačna matrica omogućila poređenje politika. U 60 validnih izvođenja završeno je svih 67.954 poslova bez greške i bez odbačenih iteracija.

Pilot je otkrio metodološku grešku u početnom statusnom anketiranju i omogućio njeno uklanjanje prije konačnih mjerenja. Stroga analiza zatim je potvrdila pet ponavljanja svake od 12 kombinacija, jednu Git reviziju i potpune artefakte.

Konačni rezultati parcijalno podržavaju H1 jer su dinamičke politike smanjile worker-minute za 14,7% do 83,4%, ali ne dokazuju stvarnu novčanu uštedu na fiksno alociranom hostu. H2 je podržana uslovno: efikasnost aktiviranih workera je povećana bez gubitka poslova, dok su zahtjevniji profili pokazali cijenu u vidu većeg čekanja i vremena obrta.

Za posmatranu aplikaciju nema univerzalno najbolje politike: KEDA najviše štedi kapacitet, CPU HPA bolje prati postepeni rast, a statička alokacija najbolje štiti latenciju. Dalji rad treba uključiti višestruke čvorove, unaprijed definisan SLO i proaktivno skaliranje.

LITERATURA

[1] Marinescu, D. C. Cloud Computing: Theory and Practice. 2nd ed. Morgan Kaufmann, Elsevier, 2018.

[2] Mell, P., Grance, T. The NIST Definition of Cloud Computing. NIST Special Publication 800-145, 2011. https://doi.org/10.6028/NIST.SP.800-145.

[3] Liu, F., Tong, J., Mao, J., Bohn, R., Messina, J., Badger, L., Leaf, D. NIST Cloud Computing Reference Architecture. NIST Special Publication 500-292, 2011. https://doi.org/10.6028/NIST.SP.500-292.

[4] Armbrust, M. et al. A View of Cloud Computing. Communications of the ACM, 53(4), 50-58, 2010. https://doi.org/10.1145/1721654.1721672.

[5] Buyya, R., Yeo, C. S., Venugopal, S., Broberg, J., Brandic, I. Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility. Future Generation Computer Systems, 25(6), 599-616, 2009. https://doi.org/10.1016/j.future.2008.12.001.

[6] Beloglazov, A., Abawajy, J., Buyya, R. Energy-aware resource allocation heuristics for efficient management of data centers for Cloud computing. Future Generation Computer Systems, 28(5), 755-768, 2012. https://doi.org/10.1016/j.future.2011.04.017.

[7] Lorido-Botran, T., Miguel-Alonso, J., Lozano, J. A. A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments. Journal of Grid Computing, 12, 559-592, 2014. https://doi.org/10.1007/s10723-014-9314-7.

[8] Al-Dhuraibi, Y., Paraiso, F., Djarallah, N., Merle, P. Elasticity in Cloud Computing: State of the Art and Research Challenges. IEEE Transactions on Services Computing, 11(2), 430-447, 2018. https://doi.org/10.1109/TSC.2017.2711009.

[9] Mao, M., Humphrey, M. Auto-scaling to minimize cost and meet application deadlines in cloud workflows. Proceedings of SC '11, Article 49, 2011. https://doi.org/10.1145/2063384.2063449.

[10] Ghodsi, A., Zaharia, M., Hindman, B., Konwinski, A., Shenker, S., Stoica, I. Dominant Resource Fairness: Fair Allocation of Multiple Resource Types. Proceedings of NSDI '11, 323-336, 2011.

[11] Zaharia, M., Borthakur, D., Sen Sarma, J., Elmeleegy, K., Shenker, S., Stoica, I. Delay Scheduling: A Simple Technique for Achieving Locality and Fairness in Cluster Scheduling. Proceedings of EuroSys '10, 265-278, 2010. https://doi.org/10.1145/1755913.1755948.

[12] Isard, M., Prabhakaran, V., Currey, J., Wieder, U., Talwar, K., Goldberg, A. Quincy: Fair Scheduling for Distributed Computing Clusters. Proceedings of SOSP '09, 261-276, 2009. https://doi.org/10.1145/1629575.1629601.

[13] Verma, A. et al. Large-scale cluster management at Google with Borg. Proceedings of EuroSys '15, Article 18, 1-17, 2015. https://doi.org/10.1145/2741948.2741964.

[14] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J. Borg, Omega, and Kubernetes. ACM Queue, 14(1), 70-93, 2016.

[15] Schwarzkopf, M., Konwinski, A., Abd-El-Malek, M., Wilkes, J. Omega: Flexible, scalable schedulers for large compute clusters. Proceedings of EuroSys '13, 351-364, 2013. https://doi.org/10.1145/2465351.2465386.

[16] Tirmazi, M. et al. Borg: The Next Generation. Proceedings of EuroSys '20, 477-495, 2020. https://doi.org/10.1145/3342195.3387517.

[17] Masanet, E., Shehabi, A., Lei, N., Smith, S., Koomey, J. Recalibrating global data center energy-use estimates. Science, 367(6481), 984-986, 2020. https://doi.org/10.1126/science.aba3758.

[18] Gupta, S., Tripathi, S. A comprehensive survey on cloud computing scheduling techniques. Multimedia Tools and Applications, 83, 53581-53634, 2024. https://doi.org/10.1007/s11042-023-17216-6.

[19] Dogani, J., Namvar, R., Khunjush, F. Auto-scaling techniques in container-based cloud and edge/fog computing: Taxonomy and survey. Computer Communications, 209, 120-150, 2023. https://doi.org/10.1016/j.comcom.2023.06.010.

[20] Kubernetes Documentation. Horizontal Pod Autoscaling. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/ (pristupljeno: 29. 7. 2026).

[21] Kubernetes Documentation. Scheduling Framework. https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/ (pristupljeno: 29. 7. 2026).

[22] Amazon Web Services. Cost Optimization Pillar - AWS Well-Architected Framework. 2026. https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/ (pristupljeno: 29. 7. 2026).

[23] Microsoft. Autoscaling Guidance, Azure Architecture Center. https://learn.microsoft.com/azure/architecture/best-practices/auto-scaling (pristupljeno: 29. 7. 2026).

DODATAK A: PRIJEDLOG ANKETNOG UPITNIKA

Ovaj upitnik je radna verzija za eventualno dopunsko istraživanje. Prije distribucije potrebno je dobiti saglasnost mentora, definisati ciljnu populaciju i način uzorkovanja te ispitanicima jasno navesti svrhu, dobrovoljnost i anonimnost učešća. Ne prikupljaju se ime, prezime ni adresa elektronske pošte.

1. Koja je Vaša primarna uloga: razvoj softvera, DevOps/SRE, sistemska administracija, cloud arhitektura, menadžment ili drugo?
2. Koliko godina iskustva imate u IT oblasti: manje od 1, 1–3, 4–6, 7–10 ili više od 10 godina?
3. Da li Vaša organizacija koristi javni, privatni ili hibridni oblak, ili ne koristi cloud infrastrukturu?
4. Koliko često u radu koristite automatizovano skaliranje: nikada, rijetko, povremeno, često ili svakodnevno?
5. Automatizovano skaliranje doprinosi boljoj iskorištenosti cloud resursa. [skala 1–5]
6. Automatizovano skaliranje doprinosi smanjenju troškova infrastrukture. [skala 1–5]
7. Nedovoljno kvalitetne metrike predstavljaju značajnu prepreku pouzdanom skaliranju. [skala 1–5]
8. Vrijeme pokretanja novih instanci ili kontejnera značajno utiče na kvalitet reakcije sistema. [skala 1–5]
9. Za odluke o skaliranju potrebno je koristiti aplikacijske metrike, a ne samo CPU i memoriju. [skala 1–5]
10. Hibridna kombinacija prediktivnog i reaktivnog skaliranja je korisnija od samo jednog pristupa. [skala 1–5]
11. Strah od narušavanja dostupnosti usluge ograničava agresivnije smanjenje kapaciteta. [skala 1–5]
12. Koje metrike najčešće koristite za odluke o alokaciji i skaliranju?
13. Koji je najveći praktični problem pri upravljanju cloud resursima u Vašem okruženju?
14. Koju biste promjenu preporučili za efikasnije upravljanje resursima?

Za tvrdnje sa skalom preporučuje se jedinstvena Likertova skala: 1 – uopće se ne slažem, 2 – uglavnom se ne slažem, 3 – niti se slažem niti se ne slažem, 4 – uglavnom se slažem, 5 – u

potpunosti se slažem. Rezultate treba prikazati deskriptivnim pokazateljima i jasno navesti ograničenja uzorka.